

SY7T108 MPU PARAMETERS

Version 3.11
January 2021



©Copyright 2021, Silergy Corp.

All rights reserved.

No part of this documentation may be reproduced nor distributed in any form or by any means, graphic, electronic, or mechanical, including but not limited to photocopying, scanning, recording, taping, e-mailing, or storing in information storage and retrieval systems without the written permission of Silergy Corp. (hereafter, "Silergy"). Products that are referenced in this document such as Microsoft Windows® may be trademarks and/or registered trademarks of their respective owners. Silergy makes no claim to these trademarks. While every precaution has been taken in the preparation of this document, individually, as a series, in whole, or in part, Silergy, the publisher, and the author assume no responsibility for errors or omissions, including any damages resulting from the express or implied application of information contained in this document or from the use of products, services, or programs that may accompany it. In no event shall Silergy, publishers, authors, or editors of this guide be liable for any loss of profit or any other commercial damage caused or alleged to have been caused directly or indirectly by this document.

Rev. 3.11, January 2021

CONTENTS

1	Introduction.....	6
1.1	Using This Document.....	6
1.2	Related Documentation.....	6
1.3	Compatibility Statement.....	6
2	Command Line Interface	8
2.1	On Line Help	8
2.2	Changing CE Parameters.....	9
2.3	Changing MPU Parameters.....	10
2.4	Changing Hardware Registers.....	10
2.5	Changing the Meter Display	11
2.6	Commanding Battery Modes.....	12
2.7	CE and Calibration Commands	13
2.8	RTM: Real Timer Monitor (of CE Variables).....	14
2.9	Autoreport.....	14
2.10	Connect Test Mux Output	15
2.11	DIO Read and Write.....	15
2.12	EEPROM Control.....	16
2.13	Error Recording and Display	16
2.14	Flash Control.....	16
2.15	Meter Information (I)	17
2.15.1	Information Fields of the I command	17
2.16	Pulse Control	18
2.17	Pulse Width Modulation.....	18
2.18	Read and Write RAM	18
2.19	Real Time Clock (RTx).....	19
2.20	Trim Operation	19
2.21	Version Information.....	20
2.22	Reset Commands (Zx).....	20
3	MPU Parameter Addresses	21
4	MPU Log (Status Bits)	34
5	Using Autocalibration.....	36
5.1	Autocalibration Procedure:	36
5.2	Modifications for Production	37
6	Command CLC.....	38
6.1	When to use it:	38
6.2	WRATE (Wh/Pulse) Calculated	38
6.3	Wh per LCD Count Calculated	38
6.4	SAG_THR (Sag Voltages) Calculated	39
6.5	CREEP_THR Calculated	39
6.6	Overcurrent Limit Calculated.....	39
6.7	Overvoltage Limit Calculated.....	39
6.8	Ix_SCALE (LSB translations) Calculated.....	39
6.9	Changing the pre-amplifier (pre_en) bit.....	39

7	Prototyping LCD operation with the Demo code	41
8	Building the Demo Code	41
8.1	For ARM's Keil ARMCC:	41
8.2	For Eclipse/GCC:	41
8.3	For Rowley's ARM GCC:	41
9	Demo Application Code	42
9.1	Demo Code Table	42
9.2	Application Code Walk-Through	44
9.2.1	Main (Demo\Src\main\main.c)	44
9.2.2	main_mission_mode_init() (Demo\Src\main\main.c)	46
9.2.3	main_mission_mode() (Demo\Src\main\main.c)	48
9.2.4	main_timing_tasks() (Demo\Src\main\main.c)	49
9.2.5	main_brownout_mode_init() (Project\Demo\Src\main\main.c)	50
9.2.6	main_brownout_mode() (Demo\Src\main\main.c)	51
9.2.7	DSP transfer done (XFER_I_N) Interrupt (\drivers\src\ce\ce.c)	52
9.2.8	meter_run() (Demo\Src\meter\meter.c)	53
9.2.9	Meter (... \library\src\meter\meter.c)	54
9.2.10	lcd_update() (Demo\Src\LCD\lcd_1phase.c)	54
10	Appendices	56
10.1	A.1 Acronyms	56
11	Revision History	58

TABLES

Table 1: On-Line Help	8
Table 2: Changing the CE Parameters	9
Table 3: Changing the MPU Firmware Calibration Parameters	10
Table 4: Changing Hardware Registers	10
Table 5: Meter Display Selection	11
Table 6: Battery Modes	12
Table 7: More CE and Calibration Control Parameters	13
Table 8: RTM Controls	14
Table 9: Autoreporting	15
Table 10: DIO Read and Write	15
Table 11: EEPROM Control	16
Table 12: Error Recording	16
Table 13: Flash Control	16
Table 14: Meter Information Command	17
Table 15: Pulse Control	18
Table 16: Pulse Width Modulation Demonstration	18
Table 17: Read and Write RAM	19
Table 18: Read and Write RAM	19

Table 19: Real Time Clock	19
Table 20: Trim Operations.....	20
Table 21: Versions	20
Table 22: Reset.....	20
Table 23: MPU Parameter Address	21
Table 24: MPU Log Bits – bits_sticky.....	34
Table 25: ARM SDK API files.....	42
Table 26: Revision History	58

FIGURES

Figure 1: System Overview	6
Figure 2: Main Loop.....	44
Figure 3: Init. Main Mission mode	46
Figure 4: Main Mission Mode	48
Figure 5: Main timing tasks.....	49
Figure 6: Brownout mode init.....	50
Figure 7: Brownout mode	51
Figure 8: Meter Run.....	53

1 Introduction

This document describes the high-level code to demonstrate the ARM Meter software development kit. This document describes only the Demo Meter Application (grey) box of the following diagram.

The SY7T M11 SDK user's guide describes the overall structure, and the drivers, the software in the orange and blue boxes.

This document starts from the user's view of the software, then describes the build, and the structure of the code.

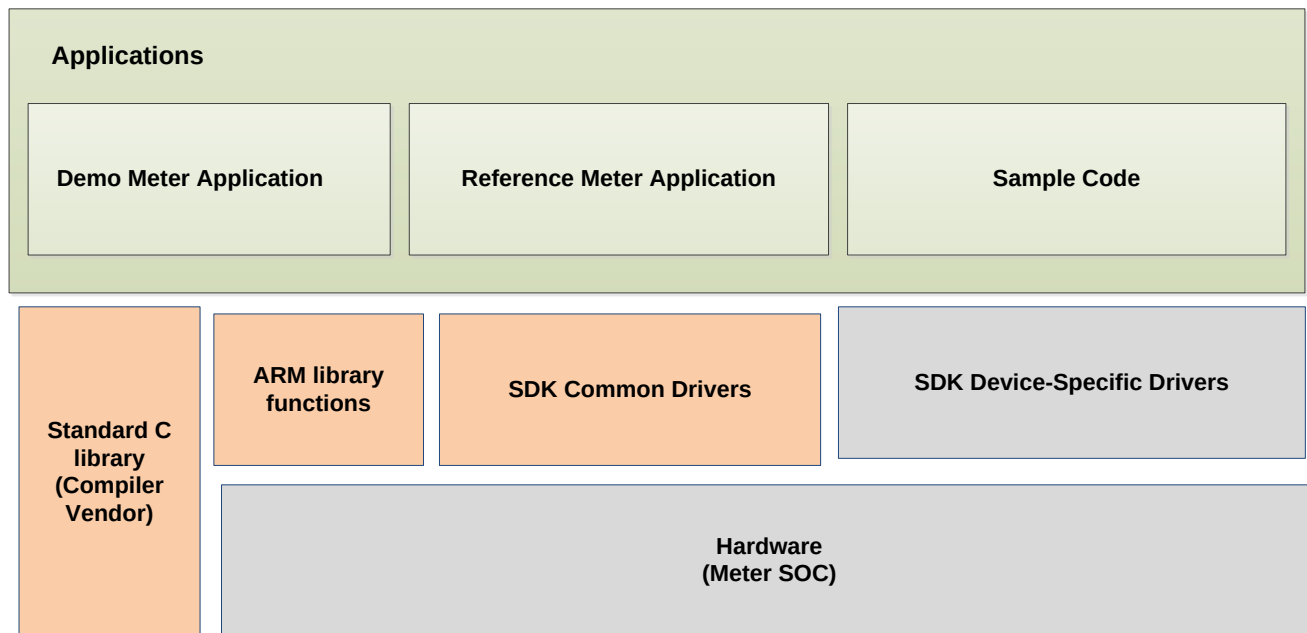


Figure 1: System Overview

The Demo code provides various resources to demonstrate the SY7T Smart metering ICs but should not be regarded as production code. The Demo code and all its components, with the exception of the CE code, are only example code and the use of it is “as is” and without implied guarantees. Demo code revisions may not be directly compatible with previously released revisions or customized software. See the Release Note for historical changes.

1.1 Using This Document

Use this document in conjunction with the SY7T M11 SDK User's Guide, and the Hardware Reference Manual when necessary.

1.2 Related Documentation

Please refer to the SY7T M11 Release Notes for a table of recommended reading.

1.3 Compatibility Statement

Information presented in this manual applies to the following hardware and software revisions:

Keil ARM C, 5.27and later.

SY7T-series devices:

- Software labelled SY7T108 is tested with part no. SY7T108G; Consult your representative for other compatibility.
- The configuration of the CE code is for single-phase.
- For each firmware release package, there is always a release note that lists a table of different revision numbers of a peripheral's driver as well as the revision number of the release package. Please review the Release note first for firmware compatibility statement and information.

2 Command Line Interface

As a quick reference, the Command Line Interface (CLI) provides access to the Demo firmware's registers as follows:

] – command to read or write CE Data RAM

) – command to read or write MPU Data RAM

? - value to read is in decimal format

\$ - value to read is in hexadecimal format

{Starting address} is in hexadecimal format only.

Other alpha character commands are as described below.

2.1 On Line Help

Table 1: On-Line Help

?	On-line help	Comment
Description:	Allows user to find the available CLI commands.	
Usage:	?[cli command prefix]	
Command combinations:	?	Get a list of help items.
	?)	Get a list of MPU parameters.
	?)	How to access CE parameters. (CE addresses are listed but CE codes vary, so the MPU code can't reliably list all CE addresses for all CE codes).
	?B	Battery mode control help.
	?C	CE help
	?CL	Calibration help
	?CR	RTM help
	?D	Read and write DIOs
	?EE	EEPROM help
	?ER	Error recording help
	?F	Flash memory help
	?I	Identification help
	?M	Meter display modes help
	?P	Pulse control help
	?PWM	Pulse width modulation help
	?RA	RAM access help
	?RM	SFR R/W access help
	?RT	RTC Control help
	?T	Trim help
	?V	Device Driver Versions
	?X	Remote registers help
	?Z	Reset command help

?	On-line help	Comment
Example:	?	See the overall help.

2.2 Changing CE Parameters

Table 2: Changing the CE Parameters

]	CE Data Access	Purpose
Description:	Allows user to read from and write to CE data space.	
Usage:	][Starting CE data address][option]...[option]	
Command combinations:	]?]	Get a list of CE interface addresses.
	]A???	Read consecutive 32-bit words in decimal, starting at hexadecimal location 0x0A.
	]A\$\$\$\$	Read consecutive 32-bit words in hex, starting at hexadecimal location 0x0A.
	]A=n=n	Write consecutive 32-bit words in hex, starting at hexadecimal location 0x0A.
	]A=+n=-n	Write consecutive signed 32-bit words in decimal, starting at hexadecimal location 0x0A.
Example:	]10???	Read CE address 0x10, 0x11, 0x12 in decimal. (These are usually the current gain, voltage gain, and phase adjustment of meter phase zero.)
	]13=4000=0=0	Writes hex values to CE address 0x13 = 0x4000, 0x14 = 0x0000, 0x15 = 0x0000. (These are usually the current gain, voltage gain and phase adjustment of meter phase 1.)

2.3 Changing MPU Parameters

Table 3: Changing the MPU Firmware Calibration Parameters

)	MPU Data Access	Comment
Description:	Allows user to read from and write to the MPU/ARM's parameters.	
Usage:	)[Starting MPU parameter number][option]...[option]	
Command combinations:	?)	Get a list of MPU calibration parameters. Note that a number of calibration parameters only take effect after running the command CLC (See 2.7, 6)
	)A???	Read 3 consecutive numbers in decimal, starting at hexadecimal location 0x0A. Note that MPU data have variable numbers of bits, and can be either signed or unsigned. Numbers will be displayed as negative if the most significant bit is set.
	)A\$\$\$	Read 3 consecutive numbers in hex, starting at hexadecimal location 0x0A. Note that MPU data have variable numbers of bits, and can be either signed or unsigned.
	)A=n=n	Write 2 consecutive numbers in hex, starting at hexadecimal location 0x0A. Note that MPU data have variable numbers of bits, and can be either signed or unsigned. Note that a number of calibration parameters only take effect after running the command CLC (See 2.7, 6)
	)A=+n=-n	Write 2 consecutive numbers in decimal, starting at hexadecimal location 0x0A. Note that MPU data have variable numbers of bits, and can be either signed or unsigned. Note that a number of calibration parameters only take effect after running the command CLC (See 2.7, 6)
Example:	)40\$\$\$	Read parameter number 0x40, 0x41, 0x42 in hexadecimal.
	)2=12345678=9876abcd	Writes 2 hexadecimal parameters starting at MPU address 0x02=0x12345678, 0x03=0x9876abcd.

2.4 Changing Hardware Registers

Table 4: Changing Hardware Registers

RM	SFR access	Comment
Description:	Allows user to read from and write the ARM's special function registers.	
Usage:	RM [register address][option]...[option]	
Command combinations:	rm2000???	Read 3 consecutive numbers in decimal, starting at register at 0x40002000. In the address, bit 30 is always set, so RM40002000 and RM2000 return the same number. Note that SFRs are memory-mapped and always have 32 bits. Decimal numbers will be displayed as negative if the most significant bit is set.
	rm4008\$\$\$	Read 3 consecutive numbers in hex, 32-bits long, starting at register address 0x40004008. Note that SFRs always have 32 bits.

	rm2000=n=n	Write 2 consecutive numbers in hex, starting at register 0x40002000. Note that SFRs have 32 bits.
	rm2000=+n=-n	Write 2 consecutive numbers in decimal, starting at register 0x40002000. Note that SFRs have 32 bits.
Example:	rmD038\$\$\$	Read SFR at 0x4000D038, 0x4000D03C, and 0x4000D040 in hexadecimal.
	rmD038=12345678=9876abcd	Writes to SFR 0x4000D038 and 0x4000D03C.

When using command “RM” command to read the register SYS->WAKE_SRC, the rtc_rd and rtc_busy bits (bits 0 and 1) are briefly set during this command to enable valid reads from shadow registers. This is “as-designed.” These bits are not set during normal operation.

2.5 Changing the Meter Display

Table 5: Meter Display Selection

M	Meter Display	Comment
Description:	Sets the meter display, and the simulated meter display performed by the CLI on the serial port.	
Usage:	M[option].[phase]	
	If a display is set, and then the calibration is saved, that display becomes the power-up default. The meter normally displays demonstration values: milliwatt-hours. For a more realistic, prototype display of Kilowatt-hours, type “)1=1<enter>” This is a calibration that can be saved. An automatically changing display of selected items is possible by setting the LCD mask. This is a configuration that can be saved. Pressing the button causes the display to step through the options.	
Command combinations:	M0	Display the meter’s serial number. (The serial number is a calibration that can be set with the “)”) command.)
	M1	Display the temperature in degrees C.
	M2	Frequency (Hz)
	M3	Accumulated imported real energy. (Wh) The factory default display. (The billing registers can be reset to zero with the command “)1=2”)
	M4	Accumulated exported real energy. (-Wh)
	M5	Accumulated inductive or imported VARh. (Volt-amps reactive)
	M6	Accumulated capacitive or exported VARh.
	M7	Accumulated apparent energy (VAh).
	M8	Elapsed hours of powered operation.
	M9,M10	Time of day, Date
	M11	Power factor
	M13	Zero crossings of mains.
	M14	Accumulated sag time.
	M15.[phase]	RMS current, 1 = Phase A, 2 = Phase B or Neutral
	M16	RMS Voltage.

M	Meter Display	Comment
	M17.[battery index]	Battery voltage, 0 = VBAT, 1 = VBAT_RTC
	M18.[phase]	Instantaneous power. The M18 Watt display can show power when the M15 Amps or M16 Volts display shows zero. High values of Volts with low Amps, or high Amps with low Volts can raise Watts above the Watts channel's noise floor. (in Drivers \Src \afe \afe*.c, see afe_watts() and afe_vacrep())
	M19.[phase]	Instantaneous volt-amps reactive (VARs).
	M20.[phase]	Instantaneous volt-amps.
	M21 M21.0	Maximum demand in W since last demand reset. (Note: The demonstration demand interval is 2 minutes, but it's an MPU calibration settable with the “)” command. The demand reset command is “)1=4”)
	M21.1	Time of day of start of maximum demand interval.
	M21.2	Date of maximum demand.
	M22	Display test
Pushbutton	In normal operation, pressing the pushbutton should cause the meter display to change to the next display. In sleep mode, it should cause the “brownout” message to be displayed. Holding down the pushbutton (SW3) causes the firmware to scroll through all the M commands' available display selection.	
LCD index	The number of the current display. Set by the “M” command. This is also a calibration value. If the display is selected with “M”, and the calibration is saved, that display becomes the default display shown after reset. This is useful to provide a prototype.	
LCD bits	An MPU calibration variable (see the command “?)” for the number). The variable contains a bit mask. The display will automatically change the display, showing the “M” display of each bit number. This is useful to provide a prototype.	
)1=1	Sets the configuration bit to enable KWh displays. Then, save the configuration. Useful for a working prototype. Not recommended for a demonstration meter; KWh meters can take hours to change their display. The Eval meter display is much faster, milliwatt-hours or centiwatt-hours, depending on the options in the LCD glass.	
)1=2	Clear the billing registers.	
)1=4	Reset the demand.	

2.6 Commanding Battery Modes

Table 6: Battery Modes

B	Battery Modes	Comment
Description:	Allows user to control battery modes. Note: The system voltage should be below 3.0V, and VBAT must be powered. This forces the firmware into brownout mode (i.e. indicated when serial prompt is “B>”)	
	To test hardware wake options, change the bits of the wake enable register using the BR command.	

B	Battery Modes	Comment
	Removing power when Vbat is powered.	Unit should enter brownout mode for a few seconds then go to sleep. If enter is pressed on the serial port, it remains awake for a further half hour, permitting engineering tests of brownout mode.
	Powering the battery, but not the main 3V power.	Unit should enter brownout mode for a few seconds then go to sleep, to preserve the battery.
	Pushbutton	In the sleep mode, pressing the pushbutton normally shows the Wh billing register for a few seconds. Internally, the unit wakes, displays the data, enters LCD mode, wakes after the timer expires, then enters sleep mode.
	Restoring main power	Unit should resume metering.
Usage:	BL	Enters LCD-only mode, a battery conservation mode that is described in the hardware reference manual. This command only operates if the demo system is running on battery power.
	BRn	Sets register SYS_WAKE to the 32-bit hexadecimal number 'n'. SYS_WAKE is a hardware register described in the hardware reference manual. Setting SYS_WAKE permits selection of novel wake configurations when testing with the system modes sleep (invoked by BS) and LCD-only (invoked by BL).
	BS	Enters sleep mode, a battery conservation mode that is described in the hardware reference manual. This command only operates if the demo system is running on battery power.
	BT	Commands a battery test. There is no visible change or serial response. The demo code runs battery test either once per hour or once per day to save battery capacity. That makes it hard to test, so there is a command to run it. The demo battery test logs a battery failure. The error log is displayed with the command ERD (See 2.13). To generate a loggable failure, the battery must be bad, i.e., VBat must have a voltage below the minimum recommended operating threshold.
	BWSn	Sets the wake timer to n seconds. Then enter BL or BS and the wake timer will wake the unit.

2.7 CE and Calibration Commands

Table 7: More CE and Calibration Control Parameters

C	Compute engine/Calibration	Comment
Description:	Allows user to configure, calibrate and control the metering.	
Usage:	C[option][arguments]	
Command combinations:	CEn	Compute engine control n ->1 Enabled (Default) n->0 Disabled

	CLS	Save calibration to EEPROM.
	CLR	Read calibration from EEPROM. If the EEPROM's calibration is invalid, substitute the factory default.
	CLD	Restore the calibration to the factory default. Calibrations return to unity gain and phase. The EEPROM's calibration data is not affected.
	CLC	In general, if any front-end MPU parameter is in engineering units (volts, amps, watts, watt-hours, etc.) it does not take effect until CLC is performed. Use this to recalculate the meter's set-up when i_max, v_max, i_min, v_min, or any other front-end parameter has been changed. Affects some CE set-up values, as well, notably WRATE, SAG_THR and I_SCALE values. (See also 6)
	CLB	Perform an automatic calibration, assuming that the meter's signal has the voltage, current and phase angle describing in the MPU parameters. (MPU parameters are described in the on-line help via "?") The calibration is automatically saved to EEPROM.
	CS	Simulated CE operation. The meter signal description is from the autocalibration variables that describe an ideal autocalibration signal, usually 240V, 5A, 45°. Pulses are intentionally disabled. (It is not possible to use this mode to calibrate a Demo meter, because the input is simulated, not real. This can be useful for testing software.)
Example:	CEO	Disables the CE, which stops metering.
	CT0.26	Make the signal CE_BUSY (signal number 0x26) appear on TMUX0.

2.8 RTM: Real Timer Monitor (of CE Variables)

RTM provides a function very like a four-trace oscilloscope, except that it shows a real-time trace of the data in up to four CE variables. CE code is delivered debugged, so customer use of RTM is usually not required.

It attaches two internal digital signals to the test mux outputs: A clock, and data. When started these send a copy of up to four CE memory cells on each sample of the analog front end.

The data is interpreted by a pod attached to a special PC with special software. RTM has demanding throughput requirements, and the software and hardware fail on many models of PCs.

Utilizing the test mux registers for LCD drivers, GPIO or other purposes is not recommended if the DSP needs to be debugged.

Table 8: RTM Controls

RTM	Purpose
CRSa.b.c.d	Control RTM Settings: Puts out CE locations a,b,c,d (hex addresses) in real time on the TMUX. This also enables the RTM (i.e. it performs CRE1).
CREn	n->1 Enable RTM n->0 Disable RTM

2.9 Autoreport

Table 9: Autoreporting

Ctrl-x, Ctrl-y	Autoreporting	Purpose
Description:	Allows user to collect comma-separated meter data on the UART.	
Usage:	Ctrl-x Starts data collection, Ctrl-y stops it	
Command combinations:	When two serial ports are available, operate data collection on one serial port, use the CLI on another.	Collect data continuously during operation. It provides a way to detect anomalies.
Data format	It sends in comma-delimited format it sends: 1. The column headers. 2. The data A typical format is: Accumulation interval index, Vrms, Irms, W, Wh, VAR, VARh, then for IB, Vrms, Irms, W, Wh, VAR, VARh.	A terminal emulator can capture the data in a file, and the file can be imported into a spreadsheet or database. This provides the data needed by most laboratory experiments on a meter. The accumulation interval is a convenient measure of time.

2.10 Connect Test Mux Output

CT	Comment
CTn.m	Select signal m on test multiplexer output n. n is 0 or 1. m is a hex number. See the Hardware Reference Manual and the PCB schematic for test multiplexer signals and pins. The test multiplexer is used for the Real Time Monitor outputs (See CRS), and can also usually show internal clocks, CE timing signals, interrupt signals and power status signals.

2.11 DIO Read and Write

Table 10: DIO Read and Write

D	DIO Read & Write	Comment
Description:	Allows user to read from and write the DIO register to configure a pin.	
Usage:	D[Starting DIO number][option]...[option]	
Command combinations:	?D	Get a list of these commands
	D0???	Read 3 consecutive DIO registers in decimal, starting at decimal location 0. DIO registers are 8 bits.
	D0\$\$\$	Read 3 consecutive DIOs in hex, starting at decimal location 0. DIO registers are 8 bits.
	D0=n=n	Write 2 consecutive DIOs in hex, starting at hexadecimal location 0x0. DIO registers are 8 bits.
Example:	D42\$	Read DIO 42 in hexadecimal.
	D42=3	Writes hex 3 to DIO 42. (i.e. make it a GPIO output, high.)
	D42=2	Writes hex 2 to DIO 42. (i.e. make it a GPIO output, low).
		Bits (0 = data, 1 = dir, 2 = alt, 3 = int. flag, 4 = pol, 5 = int. en)

2.12 EEPROM Control

Table 11: EEPROM Control

EE	EEPROM CONTROL	Comment
Description:	Allows user to read from and write to the EEPROM.	
Usage:	EE[option][arguments]	
Command combinations:	EEE	Erase the EEPROM. (Can take up to 30 seconds. Erases calibration data, error data, and billing register data.) Note: This does not erase the RAM copies. A hardware reset is required to erase the RAM data.
	EECn	n->1 Enable EEPROM, n->0 disable EEPROM.
	CLD	Load default calibration.
	CLS	Saves the calibration to the EEPROM.
	)1=2	Empty billing registers, including the EEPROM copies of them.
	EERa.b	Read from EEPROM at hex location a, for decimal length b bytes.
	EESabc...xyc	Write characters to a buffer.
	EETa	Transmit the buffer to the EEPROM at hex address "a".
	EEWab...z	Write hex or decimal 8-bit bytes to the buffer.
Example:	EEShello	Writes "hello" in ASCII to the buffer then write buffer to EEPROM at
	EET2	location 2.

2.13 Error Recording and Display

Table 12: Error Recording

ER	Error recording and display	Comment
Description:	Allows user to read meter's log of events and errors.	
Usage:	ER[option][arguments]	
Command combinations:	ERD	Error read. Error numbers correspond to bit numbers in a nonvolatile sticky-bit copy of the status word. This code interprets the bits, giving current messages.
	ERC	Clear all errors and resense. Does not perform start-up tests. Use 'Z' to reset the meter and resense.
	ERSn	Set error No. n (0 <= n <= 31)
	ERUn	Clear error No. n

2.14 Flash Control

Table 13: Flash Control

F	FLASH CONTROL	Comment
Description:	Allows user to read from and write to the flash.	
Usage:	F[option][arguments]	
Command combinations:	FEa	Erase flash page at address a. Where 'a' is a 32-bit aligned byte address .

F	FLASH CONTROL	Comment
		Any address within a flash page is acceptable.
	FRa.b	Read flash long-word(32-bit) starting at 'a' for 'b' long-words.
	FTa	Transmit the buffer to the flash at hex address "a". Note: If a destination word is not 0xffffffff writes to it will fail.
	FWa.b.c	Stores a buffer of long-words (e.g. 0x0a, 0x0b, 0x0c), sets write length (e.g. 3). They can then be written by "FT".
Example:	FE3F000 FWffff0000.eeee0000 FT3f000 FR3f000.2	Erases flash page at 3F000. Writes 0xffff0000, 0xeeee0000 to the flash long-words at location 3F000 and then reads it back.

2.15 Meter Information (I)

Table 14: Meter Information Command

I	Information messages	Comment
Description:	Allows user to read information about the meter.	
Command combinations:	I	Identifies the meter's firmware and hardware on the serial port. If the firmware is built to handle remotes, it prints a list of remotes. The JEDEC ID fields (JEP106 continuation codes, ID, part number, major and minor revisions) are displayed in line #2.
	MO	Identifies the meter's serial number (a configuration saved with the calibration) on the display.

2.15.1 Information Fields of the I command

A typical output is as follows:

```
SY7T108C0 CEM11a02e v3.0.0 SHUNT-CT 1P2W 20:00:11 Jun 14 2020
Chip ID:0x0000 Rev.:1 jep106 id:0x58 continuation:10 (c) Silergy Corp.
```

The top line summarizes some human-readable fields of the system:

1. SY7T108 shows the family of IC. It gives some assurance of software compatibility. It does not define RAM or flash sizes. The family name can be associated with different ICs that vary in other ways to meet local conditions or regulations. Use a data sheet to select an exact IC.
2. C0 is a version of the associated ICs. It might describe process changes as well as register changes. It is read from a numeric device register and translated to a letter and digit.
3. CEM11 describes the type of CE required by this firmware.
4. a02e further identifies the CE code. In general, CE codes are not interchangeable and have different reference manuals and set-up code.
5. V3.0.0 is a three part version number for the firmware. The first version number, e.g. "3" is usually associated with the IC version, i.e. version C. The second version changes with major

firmware requirements. The third number changes with minor improvements and correction of defects. Unreleased internal versions for testing are usually distinguished only by compile date.

6. SHUNT_CT describes the current sensor for measurement, and for neutral.
7. 1P2W describes the meter drop equation, 1 Phase, 2 Wires, i.e. $W = VA * IA$
8. 20:00:11 Jun 14 2020 describes a compile time and date.

The second line prints some machine-readable ID fields of the IC, taken from the standardized ARM Peripheral ID registers. These are sometimes used by debuggers or software to self-configure.

9. Chip Id:0x0000 is a hexadecimal print of a 16-bit Silergy part number.

0xf0000fe4 [3:0] = chip_id[15:12], ARM Part No.[11:8], from ARM register ETMPIDR1

0xf0000fe0 [7:0] = chip_id[11:4], ARM Part No.[7:0], from ARM register ETMPIDR0

0xf0000fe8 [7:4] = chip_id[3:0], ARM Revision, from ARM register ETMPIDR2

10. Rev.: The ARM ECO (or “minor”) revision as determined by the manufacturer.

0xf0000fec [7:4] = chip revision, ARM ECO Revision, from ARM register ETMPIDR3

11. jep106 ID: 0x58 is a manufacturer ID based on JEDEC standard jep106. 0x58 is Silergy’s ID.

0xf0000fe4 [7:4] = JEP106 Company ID Code[3:0], from ARM register ETMPIDR1

0xf0000fe8 [3] = Should be 1, from ARM register ETMPIDR2

0xf0000fe8 [2:0] = JEP106 Company ID Code[6:4], from ARM register ETMPIDR2

12. “continuation:10” Silergy’s name is found in the 10th continuation page of ids in the JEDEC jep106 standard.

0xf0000fd0 [3:0] = JEP106 company ID code continuation count, from ARM register ETMPIDR4

2.16 Pulse Control

Table 15: Pulse Control

P	Load control	Comment
Description:	Allows user to set CE variable for output by WPULSE	
Usage:	P0 P[hex address of CE variable]	
Example:	P0 - Pulse billable sum of Whr (default). P85 – Pulse using data in CE location hex 0x85.	

2.17Pulse Width Modulation

Table 16: Pulse Width Modulation Demonstration

PWM	Load control	Comment
Description:	Allows user to test pulse width modulation	
Usage:	PWM[channel].[Hertz].[percent]	
Example:	PWM1.1000.50 - Starts PWM1 (beeper) at 1000Hz and 50% duty cycle. PWM1.0.0 - Stops PWM1.	

2.18 Read and Write RAM

Table 17: Read and Write RAM

RA	RAM access	Comment
Description:	Allows user to read from and write the ARM's RAM.	
Usage:	RA[address][option]...[option]	
Command combinations:	ra20000800???	Read 3 consecutive numbers in decimal, starting at register at 0x20000800. Note that CE RAM starts at 0x20000000, MPU RAM about 1.5K later. Decimal numbers will be displayed as negative if the most significant bit is set.
	ra20000800\$\$\$	Read 3 consecutive numbers in hex, 32-bits long, starting at register address 0x20000800.
	ra20000800=n=n	Write 2 consecutive numbers in hex, starting at register 0x20000800.
	ra20000800=+n=-n	Write 2 consecutive numbers in decimal, starting at register 0x20000800.
Example:	ra200008e0\$\$\$	Read at 0x200008e0 in hexadecimal.
	ra200008e0=12345678=9876abcd	Writes to 0x200008e0=0x12345678, 0x200008e4=0x9876abcd in hexadecimal.

Table 18: Read and Write RAM

2.19 Real Time Clock (RTx)

Table 19: Real Time Clock

RT	Real-Time Clock	Comment
Description:	Control and display the meter's clock	
Usage:	RT[option][value]...[value]	
Command combinations:	RTR	Read the real-time clock in local time.
	RTZ	Read the real-time clock in universal time. (The time ARM is set in the MPU parameters by the number of minutes offset from GMT. The factory default is Pacific Standard Time.)
	M9	Causes a clock display on the LCD and serial simulated LCD.
	M10	Causes a date display on the LCD and serial simulated LCD.
	RTDy.m.d	Sets the date: year, month (1=January), day of month. The day of week and day of year are calculated.
	RTMh.m	Sets the hardware alarm hours and minutes. When alarm goes off, it logs an RTC alarm, visible with command "ERD".
	RTT.h.m.s.dst	Sets the time of day, with hour, minute, second (defaults to 0 if omitted) and daylight savings time (1 = daylight savings time, 0 = standard time. Defaults to standard time if omitted).
	RTU	Unlocks real time clock, if locked by error prevention logic.

2.20 Trim Operation

Table 20: Trim Operations

T	Trim operations	Comment
Description:	Trim read, erase and write.	
Usage:	T[option][parameter]	
Command:	TR[1...40]?[?...]	Read trims in decimal
	TR[1...40]\${\$...}	Read trims in hex

2.21 Version Information

Table 21: Versions

V	Versions	Comment
Description:	Reads versions of software files.	
Usage:	V[option]	
Command:	V,V0,V1	Versions of common drivers, i.e. drivers shared by most models of the SY7 family.
	V2	Versions of drivers for a particular model of SoC.
	V3	Versions of library files.

2.22 Reset Commands (Zx)

Table 22: Reset

Z	Reset testing	Comment
Description:	Performs resets in all available ways	
Usage:	Z[option]	
Command combinations:	Z	Hardware reset of ARM M0 core only, caused by the ARM NVIC's reset request feature. The emulator is not reset.
	Z0	Hardware reset, caused by SYS->MOD_CNTL_b.reset= 1; The emulator is reset.
	Z1	Hardware reset, caused by SYS->MOD_CNTL_b.reset= 1; The emulator is reset.
	Z2	Watchdog reset, caused by a loop. If no reset, try disabling the debug interface (Remove jumper JTAG_E). The emulator is reset.
	Z3	Best-effort soft reset, done by forcing the firmware to run the reset vector. The emulator is not reset.
	Z4	Hard fault. Write to an error address in the APB test slave by setting 0x40000B0FC=0.

3 MPU Parameter Addresses

In the Demo code certain MPU RAM parameters have been given addresses in order to permit easy external access. These variables can be read in hex via the command line interface, with the **)n\$** command and written with the **)n=xx** command where n is the word address and xx is a hex number. Note that accumulation variables are 64 bits long and are accessed with **)n\$\$** (read) and **)n=hh=ll** (write) in the case of accumulation variables.

- The first part is calibrations, numbers that may need individual adjustments for demonstration.
- The second part)20... is measurements, numbers that may need to be read in a demonstration meter. Items from 1F backward are used to verify measurements.

Table 23: MPU Parameter Address

Name	Purpose	LSB	Default	)?	Signed?	Bits
i_min ¹	Metering element enters creep mode if current is below this value. If 0, creep logic is disabled. In creep mode, on each metering element, Wh, VARh, iOsqsum, and other items are zeroed.	Milliamps (0.001A)	See the #define I_MIN value in the respective <i>Demo/Include/mapeb*.h</i> file. Typically 1/2000 of i_max.	)0	signed	16
cfg	Configure meter operation on the fly.	This field can be saved as part of a calibration, to prototype a meter. Bit 0: 1=Display KWh. Bit 1: 1=clear accumulators, errors, etc. (e.g. ")1=2") Bit 2: 1=Reset demand. (e.g. ")1=4") Bit 3: RFU (reserved for future use) Some firmware may use it to provide more digits of accuracy on the	0 Do nothing special.	)1	N/A	8

Name	Purpose	LSB	Default	)?	Signed?	Bits
		displays, or show RAW CE values. Bit 4: RFU Bit 5: 1=test temperature compensation, i.e. temp_c, below becomes an input setting the system's temperature for compensation. Bit 6: 1=Auto calibration mode¹ This bit field is part of a calibration, used to configure a prototype or set-up for battery mode. This can be saved as part of a calibration. Bit 7: RFU (Some firmwares may use it to enable tamper detection.)				
v_min ¹	Below this, low voltage seconds are counted. Voltage, Wh, VARh, Frequency, and other voltage-dependent items are zeroed.	0.1 V	See the #define V_MIN value in the respective mapeb*.h file. Often 80V or 100V.	)2	signed	16
i_max ¹	Scaling Maximum Amps for standard sensor.	0.1A	See the #define I_MAXA value in the <i>Demo/Include/mapeb*.h</i> file. Based on sensor configuration.	)3	signed	16

Name	Purpose	LSB	Default	)?	Signed?	Bits
v_max ¹	Scaling Maximum Volts for Evaluation Board rev 1	0.1V	See the #define V_MAX value in the <i>Demo/Include/mapeb*.h</i> file. Based on sensor configuration.	)4	signed	16
wh_whpp	watt-hours of watt-hours per pulse.	1Wh	1 (i.e. 1 Wh/P)	)5	signed	16
p_whpp ¹	Pulse count of watt-hours per pulse.	1 pulse	1 (i.e. 1 Wh/P)	)6	signed	16
i_limit ¹	Error if exceeded.	0.1A	See the #define I_LIM value in the mapeb*.h file. Typically 95% of I_MAX. Per phase. Based on sensor configuration.	)7	signed	16
v_limit ¹	Error if exceeded.	0.1V	See the #define V_LIM value in the <i>Demo/Include/mapeb*.h</i> file. Typically V_MAX + 20%. Per phase. Based on sensor configuration.	)8	signed	16
i_max_neutral ¹	Scaling Maximum Amps for the neutral current. If the CE implements an Ix_SCALE, it is updated.	0.1A	See the #define I_MAXN value in the mapeb*.h file. Based on sensor configuration.	)9	signed	16

Name	Purpose	LSB	Default	)?	Signed?	Bits
s_cal ²	Accumulation intervals of Autocalibration	Count of accumulation intervals of calibration.	2 accumulation intervals (covers both chop polarities.)	)A	signed	16
v_cal ²	Volts of Autocalibration	0.1V rms of AC signal applied to all elements during calibration.	2400 240V is a standard full-scale set-up for meter test.	)B	signed	16
i_cal ²	Amps of Autocalibration	0.1A rms of AC signal applied to all elements during calibration. Power factor of calibration signal must be 1. Also see “theta_cal” at)1A	50 (5.0 A)	)C	signed	16
theta_cal ²	Phase angle of autocalibration signal. Permitted to be nonzero so current sensor phase calibration can have a significant nonzero measurement.	Phase angle of delay between voltage and current of calibration signal, in degrees. 45 degrees gives the maximum of both the real and imaginary vectors of the calibration signal, permitting high quality calibration of both. 60 works well.	45°	)D	signed	16
lcd_idx	Selects LCD's current display.	0: Meter identification. (“#”) 1: Display variation from calibration temperature, 0.1C 2: Display mains Hz, 0.1 Hz 3: Wh, total 4: Wh total exported. 5: VARh, total. 6: VARh, total exported.	3 (Wh)	*)E	unsigned	16

Name	Purpose	LSB	Default	)?	Signed?	Bits
		7: VAh, total 8: Operating hours. 9: Time of day 10: Calendar date 11: Power factor, total 13:Main edge count, last accumulation. 14: Seconds , accumulated sag time 114: Seconds, tamper (- = tamper in progress) ("TS") 15: RMS current, maximum of line or neutral 115: RMS current, line 215:RMS current, neutral 16: RMS voltage 18: Momentary power, W 118: Momentary power, W, line 218: Momentary power, W, neutral 19: Momentary VARs 119: Momentary VARs, L1 219: LMomentary VARs, Neutral 20: Momentary VAs 120: Momentary VAs, L1 220: LMomentary VAs, Neutral 21: Demand, W 121: Demand date 221: Demand time 22: LCD Test				
lcd_bit	Defines sequence of LCD displays.	The value is a bit mask that describes a scrolling display sequence. Each set bit causes a display. The bit number that is set, causes the	0	)F	unsigned	32

Name	Purpose	LSB	Default	)?	Signed?	Bits
		corresponding display number to appear. Each is displayed for 7 seconds. Ordered by increasing bit number. If value is zero, display does not change. E.g. 9 will alternate between the meter ID (display 0, because bit 0 is set) and Wh (display 3, because bit 3 is set) This can be saved as part of a calibration, to prototype a meter.				
mfr_id	MFR ID of meter	3 ASCII bytes, in MSB of 32-bit number. Least significant byte should be zero. For AMR demonstrations, sent as the manufacturer's ID of the meter.	"SLG", 0x534C4700	)10	unsigned	32
meter_id	Identification number of meter.	32 bit unsigned number. For AMR demonstrations, this is sent in decimal as the identification number of the meter, or used to generate the address. Can be changed and saved as part of calibration.	100000	)11	unsigned	32
gmt_offset	Used to make GMT from the RTC, which is always in local time. This takes effect only when the time is set.	Minutes that the meter is ahead or behind GMT	-480 (GMT – 8 hours, Los Angeles time)	)12	signed	16
tcab	RTC adjustment register - TCAB. Used to adjust for the quadratic crystal behavior.	See the Hardware Reference Manual.	0	)13	signed	32

Name	Purpose	LSB	Default	)?	Signed?	Bits
	Crystals are fastest near their center temperature, so this arranges the RTC to run faster at temperature extremes.					
tccd	RTC adjustment register -TCCD.	See the Hardware Reference Manual.	0	)14	signed	32
RFU				)15		
cal_storage_info	Storage info. of calibration parameter; i.e. 0=default, 1=read from EEPROM or flash, 2=from user, etc. It is a read_only parameter, writing to it will not change the system state.	Indicate the storage source of calibration parameters.	0=Default	)16	unsigned	8
cal_cnt	Count of calibrations.	Counts number of times calibration is saved, to a maximum of 255.	0	)17	unsigned	8
ver_hash	Hash of copyright message is stored in calibration data. Hash includes the FW compilation time and date and calibration data. The MPU validates compatibility between the compiled time hash and the current hash. When this value is 0xFF, it means the calibration data is default value. Hash is calculated	Usually a linear parity of the version string.	Varies with date and time of compilation.	)18	unsigned	8

Name	Purpose	LSB	Default	)?	Signed?	Bits
	when cal save is called. If the stored hash is different from current hash, the calibration data is set to defaults. It is a read_only parameter, writing to it will not change the system state.					
RFU				)19		
pulse_src	Selects source of wpulse output.	CE address of pulse output value.	CE address of WSUM_X.	)1A	unsigned	16
interval	The number of minutes of a demand interval. Note that demand intervals start on the minute of the hour that is evenly divisible by interval. Interval is designed to be 2, 15 or 30. Numbers that do not divide evenly into 60 will cause a small “remainder” interval to be measured at the end of the hour..	Count of minutes. $(60/\text{interval}) * \text{interval} = 60$.	2 minutes.	)1B	unsigned	8
RFU				)1C		
wsum_per_cnt ¹	Counts of WSUM (i.e. from CE watt field) per LCD count. Used to update display. It is a read_only parameter, writing to it will not change the system state.	Same as WOSUM_X of CE code.	Varies depending on CE's LSB.	)1D	Signed	32
v_nom ¹	Nominal voltage. In tamper-resistant	0.1V	2400 (240V)	)1E	Signed	16

Name	Purpose	LSB	Default	)?	Signed?	Bits
	meters, this is to be multiplied with the current to calculate watt-hours.					
acc_cnt	Count of accumulation intervals detected since reset. Can be used to detect lost interrupts and diagnose other firmware errors.	Count	n/a	)1F	Unsigned	32
bits_sticky	Bit array of errors and state.	See table in next section.	n/a	)20	Unsigned	32
Wh_cnt	Count of watt-hour pulses. Normally turns over from 999,999,999 to 0.	1 Wh	n/a	)21	Signed	32
wh_cnt_ex	Count of exported watt-hour pulses. Normally turns over from 999,999,999 to 0.	1 Wh	n/a	)22	Signed	32
Varh_cnt	Count of inductive (lagging, positive) var-hour pulses. Normally turns over from 999,999,999 to 0.	1 VARh	n/a	)23	Signed	32
varh_cnt_ex	Count of capacitive (leading, negative) var-hour pulses. Normally turns over from 999,999,999 to 0.	1 VARh	n/a	)24	Signed	32
dmd_max	Maximum demand	Watts	n/a	)25	Signed	32
dmd_time	Date and time starting the period of maximum demand.	Month, day of month, hour, min; On Month, 0 = January.	n/a	)26 – 29	signed	8
tamper_sec	Total time tamper was detected.	Seconds	n/a	)2A	signed	32

Name	Purpose	LSB	Default	)?	Signed?	Bits
	Display (not this register) turns over from 99,999 hours to zero. Continues to accumulate if any tamper, e.g. tamper switches are on, I without V, etc. To stop tampers, check logged tamper conditions with command erd.					
sag_sec	Total time any voltage was below v_min. Display (not this register) turns over from 99,999 hours to zero.	Seconds	n/a	)2B	signed	32
operating_sec	Total time meter has been powered. Display (not this register) turns over from 99,999 hours to zero.	Seconds	n/a	)2C	signed	32
reg_cnt	Count of saves of billing registers. Turns over from 255 to 0.	count	n/a	)2D	signed	8
ver_hash	Similar functionality as '18' but this one is stored along with the billing registers. Hash of copyright message stored in billing register data, including the time and date so firmware will ignore data not in	Usually a linear parity of the version string.	Varies with date and time of compilation.	)2E	unsigned	8

Name	Purpose	LSB	Default	)?	Signed?	Bits
	its own data format. If different from current hash, this copy of billing data is ignored. If all copies of the billing data are invalid or if CRC of the billing register is bad then the billing data is set to defaults, and an error is posted in the state. It is a read_only parameter, writing to it will not change the system state.					
data_ok_reg	CRC of global data			)2F	unsigned	8
last_wake_src	Image of register SYS->WAKE_SRC as of the most recent reset or wake.	Same as 'RMA050\$' command (bits 31:3).	n/a	)30	unsigned	32
last_wake	Image of register SYS->WAKE as of the most recent reset or wake.	Same as 'RMA03C\$' command.	n/a	)31	unsigned	32
temp_c	Temperature of SOC's bandgap reference, used to compensate the voltage reference for voltage measurements. If the correct CFG bit is set, this value becomes writable to test Vref temperature compensation without an oven.	0.1C	n/a	)32	signed	16
ppmc[0..2]	This is the linear term of the	See CE reference manual	Calculated	)33..35	signed	32

Name	Purpose	LSB	Default	)?	Signed?	Bits
	quadratic integer polynomials that take temperature and adjust the gains of meter sensors. Assignment of channels is determined by the CE code's design, which uses the GAIN_ADJ array. It varies by the location of the voltage reference for the ADCs. CT: Voltage and current are both adjusted by ppmc[0] for all voltage and current channels. This is because they all use the ARM's internal voltage reference. Single-channel remotes: All voltage sensors and neutral current are adjusted by ppmc[0]. ppmc[1]..ppmc[2] adjust the remotes for phase A and B.					
ppmc2[0..2]	This is the quadratic term of the quadratic integer polynomials that take temperature and adjust the gains of meter sensors.	See CE reference manual	Calculated	)36..38	signed	32

Name	Purpose	LSB	Default	)?	Signed?	Bits
	Assignment of channels is determined as for ppmc[], above.					

¹The parameter does not take effect until command CLC runs, or a calibration is saved and restored.

² Valid only when autocalibration is integrated.

4 MPU Log (Status Bits)

Discussion of the bits in the variable `bits_sticky`.

Nonvolatile errors are cleared when the registers are cleared, e.g. by the command “)1=2”

Higher-numbered errors are generally more severe or more interesting.

Errors with a number less than “software” are status, cleared at reset by the log module.

In the software, this list is in `projects\<project>\include\log_events.h`; It is very easy to reorder (changing priorities) or add another code. The output strings and data are in `cli.c`, so that they take no space in optimized production code.

Table 24: MPU Log Bits – `bits_sticky`

NAME:	Bit:	Description:
EV_NONE	0	None, no error.
EV_LINE_BAD	1	The billing data is taken from neutral. (This is either a meter failure or a tamper.)
EV_COM	2	Communication occurred.
EV_RTC_ALARM	3	The RTC alarm clock went off. Asserted for one minute after the alarm.
EV_TEMP_ALARM	4	Temperature out-of-range alarm occurred.
EV_SOFTWARE	5	A software defect was detected. (<code>log_software()</code> was called.) Events lower than this in priority are not preserved after a reset.
EV_WATCHDOG	6	A watchdog reset was detected: <code>SYS->WAKE_SRC_b.wf_wdof</code> was set when the firmware started.
EV_OVERVOLTAGE	7	Maximum voltage has been above <code>v_limit</code> . Meter insulation may have broken down.
EV_OVERCURRENT	8	Total current has been above <code>i_limit</code> . Meter may have overheated.
EV_HARD_FAULT	9	A hard fault occurred. Firmware continues to run.
EV_HARDWARE	10	A hardware error occurred. Firmware continues to run.
EV_SPURIOUS	11	An unexpected interrupt occurred.
EV_RTC_CORRUPT	12	RTC was found corrupted after a reset, and was reset to the value saved with the most recent copy of the billing data.
EV_VLCT_BAD	13	The voltage into the voltage line-cut pin is or has been too low to be useful.
EV_CAL_BAD	14	Set after reset if the read of the calibration data has a bad CRC, or is from an earlier version of software. The default values should be present, mitigating this problem.
EV_BAT_BAD	15	The battery voltage is or has been too low to be useful.
EV_SAG	16	At least one phase voltage is below <code>v_min</code> . Possible bad utility service or bad installation. Standard eval code has a nonvolatile count of sagged seconds.
EV_RTC_UNSET	17	The clock needs to be set. A) Obtained after a cold or battery-on start, indicating that there was no battery power, and therefore the clock has to be invalid. B) Impossible date. In cases A) & B), Eval code sets the clock to 01:01, 1/1/01. C) More than a year after the previously saved reading, D) Earlier than the previously saved reading. In cases C & D, the clock's time is preserved, but the clock should not be trusted. For all cases, production meter code should set the RTC from a date and time recorded with billing data, and continue.
EV_DEBUGGER	18	Debugger was detected.
EV_RTC_TAMPER	19	Clock set to a new value more than two hours from the previous value.

NAME:	Bit:	Description:
EV_MAG_TAMPER	20	Magnetic tamper was detected. (Unused- the Eval PCB has no magnetic sensor)
EV_I_NO_V	21	At least one phase has current but no voltage. (This is either a meter failure or a tamper.)
EV_TERM_TAMPER	22	Tamper was detected. Terminal-block open tampering condition.
EV_CASE TAMPER	23	Tamper was detected. Case open tampering condition.
EV_REG_BAD	24	Set after reset if the read of the billing register data has a bad CRC or bad software version in all copies.
EV_LOG_BAD	25	Error data had a bad CRC and was cleared, losing old errors.
RFU	26-31	Reserved for future use. (never reported)

5 Using Autocalibration

This Eval code supports an autocalibration system.

Autocalibration adjusts the meter to correctly measure a calibration signal. The Eval autocalibration is a high-quality one-point calibration that adjusts the gains for voltage and current, and the phase relation between them, to compensate for delays in the current or voltage sensors. In many autocalibrations, the phase adjustment measures and absorbs frequency errors between the meter and calibrator, using the calibrator as a frequency reference.

The meter's accuracy after autocalibration is typically within 0.1%, but never more than 0.5% with correct autocalibration code. The remaining errors appear to be caused by quantization error in the calibration quantities, round-off error in the calculations and possibly temperature changes during the calibration process.

The qualities of the calibration signal are described by setting MPU variables:

)b is the phase voltage in 0.1V (e.g. 2400 is 240.0 Volts.) This should be the nominal operating voltage.

)c is the phase current in 0.1A (e.g. 50 is 5.0 Amps). A larger current reduces the signal to noise errors. A smaller current causes less current-based distortions, e.g. intermodulation. A smaller current is also more typical for a meter in the field. 5A is about 5% of the i_max of most Eval meters, and gives good results.

)d is the phase angle, voltage to current, in degrees. A 45-degree signal maximizes the signal of both Wh and VARh, maximizing the signal to noise ratio of the phase error of the current and voltage sensors. This permits the calibration to measure the phase error more accurately and correct it more exactly. 60, 120, 240 and 300 degrees are easier for some calibrators to produce and these phase angles should give good results, although not all Eval codes have been tested with them. 0, 90, 180 and 270 degrees give poor results.

)a sets the calibration measurement time. Although longer times average more data, with some calibrators this seems to reduce accuracy. The minimum calibration time is the time to let the CE's digital filters settle (one second may work, but most systems use two seconds for safety), and then an additional time (the time set by)a) long enough for the calibration to average all possible analog chop states. (The usual default is two seconds beyond the settling time.)

5.1 Autocalibration Procedure:

1. Start the test signal.
2. Set the firmware's description of the expected qualities of the test signal, using the command line interface, e.g. for a phase with 240V, 5A and 45 degrees:

```
)b=+2400=+50=+45
```

3. Set the CE's gain and phase adjustments to defaults:

```
CLD
```

Or, set the gains and phases to the CE's defaults. E.g. for a 2-phase SY7T-series using:

```
]10=4000=4000=0=4000=4000=0=4000=4000=0
```

4. Type the calibration command:

```
CLB
```

Variations:

The autocal can adjust all the phases at once, or one at a time.

Autocal, command CLB, adjusts only default CE calibration values. This is as designed, so that several phases can be individually calibrated: The test signal can be applied, the calibration performed, and then that

phase's CE calibration values are no longer defaults, and are not adjusted by autocal after that. To adjust all of the calibration values, use CLD to set all CE calibration values to defaults. CLB will then adjust all of them.

5.2 Modifications for Production

Autocalibration is already designed to calibrate all phases at once. Since the Silergy meter system has a single calibration range, individual meter calibration can be very fast. The meters should be soaked to be at the calibration temperature, but the designed calibration temperature is room temperature, so many meter designs can be calibrated in an ordinary air-conditioned building.

A significant advantage of autocalibration is that no communication is needed between the calibrator and meter. This can simplify procedures and calibrators.

If the calibration presents test signals on all phases at the same time, the autocal code can set the gain and phase defaults automatically. Most autocal codes have commented-out code for this. The calibration description can also be modified so its default describes the standard production calibration signal. Then, just command "CLB".

Some customers find it convenient to have the meter automatically invoke autocalibration in production. To do this in the Eval code, set up the meter for single-command all-phase calibration, as above. Then add code to have the main loop examine the calibration source variable. If the calibration source is "factory defaults" (i.e. never calibrated), then if the meter voltage, current and phase is in range (e.g. within 20% of the expected signal), start the autocalibration. Using this system, to recalibrate a meter, erase the EEPROM (e.g. as if with the CLI command EEE,) and reapply the calibration signal.

One-phase two-wire meters may need to calibrate sensors for neutral current. The autocal for 1p2w (i.e. equation 0) can be changed to have two phases (i.e. for neutral metering), and adjust both for gain and phase.

6 Command CLC

Command CLC automatically recalculates Eval meter configuration variables that depend on the ADC scaling variables `i_max` or `v_max`. It should not change calibration values. Usually this is convenient, but CLC doesn't change one thing: It changes variables for pulse-rate, LCD-display/billing rate, creep, sag, over current, over voltage and translation from one current sensor to another.

Note: Changes to creep and sag values do not take effect until the command CLC is run, or the calibration is saved by CLS and a reset occurs. The `pre_en` (bit 8 of the SFR `ADC_VREF_CNTL` at `0x4000_D020`) is another bit that requires the recalculation of the `i_max`, which mandates the CLC command as a follow-up step to fully complete meter configuration variables. As designed: Internal creep values in the CE code's native units are calculated from `v_min`, `i_min`, `v_max`, `i_max` and CE constants. CLC tells the code that all the variables are done, and there are no more changes

6.1 When to use it:

Use command CLC after changing the ADC scaling in `v_max`, `i_max`, creep values in `i_min` or `v_min`, or the Wh per pulse, in `wh_whpp` and `p_whpp`.

Its original purpose was to recalculate the CE's pulse variable, `WRATE`, when `i_max` or `v_max` changed.

6.2 WRATE (Wh/Pulse) Calculated

This reads the MPU variables `wh_whpp`, `p_whpp`, `i_max`, `v_max` and the CE LSB constants, and sets `WRATE` in the CE.

`WRATE` and `wsum_per_cnt` are calibration values that can be saved, and can be modified and saved. If there is no saved calibration, they are calculated to make the default. If there is a saved calibration, they are not automatically calculated at start.

The pulse selection logic has no logic to selectively adjust values from IB, if IB has a different LSB, and the CE is not adjusting the LSB. In this case the LCD might show the correct value, yet the pulse will be wrong.

The MPU's pulse selection logic has no logic to select special creep logic for CE variables selected as a pulse source. E.g. if the MPU's pulse source variable is set to the CE's `I1SQSUM_X` the MPU firmware uses the default creep logic, with the creep limit calculated from the minimum current and voltage. One work-around is to disable the creep logic with the command `J0=0`

In all codes, for example to recalculate the Wh/pulse to 0.0533 Wh/pulse, first set the Wh/pulse: `J5=+533=+10000` then run command CLC. This sets the fraction Wh/pulse to 533 Wh / 10000 pulses, i.e. 0.0533. Most fractions will work, but the CE code's slow pulse and fast pulse bits in `CECONFIG`, `J20`, might also need to change. Check the value of `WRATE` with command `J21`? If less than 150 or more than 32767, the CE code cannot work.

6.3 Wh per LCD Count Calculated

This reads the MPU variables `i_max`, `v_max`, the PCB constant `WH_PLCDCNT` and the CE LSB constants, and sets the global MPU variable `wsum_per_cnt`, the number of CE counts from `WxSUM_X` or `VARxSUM_X` that make `LCD_WHPCNT` Wh.

6.4 SAG_THR (Sag Voltages) Calculated

This reads MPU variables `v_max` and `v_min`, and sets the CE variable `SAG_THR` in the CE (to a value equal to mpu variable `v_min`), as well as any auxiliary CE sag values (set as hard-coded percentages of `v_min`).

`SAG_THR` is a calibration value that can be saved, and can be modified and saved. If there is no saved calibration, it is calculated to make the default. If there is a saved calibration, it is not automatically calculated at start.

6.5 CREEP_THR Calculated

Some CE codes have a creep threshold variable `CREEP_THR` used with internal pulse generation, and enabled by setting bits in CE variable `CE_CONFIG`.

This calculation reads MPU variables `v_max`, `i_max`, `v_min` and `i_min`, and sets `CREEP_THR` in the CE to a VA value equal to `v_min` times `i_min`.

`CREEP_THR` is a calibration value that can be saved, and can be modified and saved. If there is no saved calibration, it is calculated to make the default. If there is a saved calibration, it is not automatically calculated at start.

6.6 Overcurrent Limit Calculated

It reads MPU variables `i_lim`, `i_max` and CE LSBs and sets internal MPU variables that are compared against the CE's currents.

The internal variables are recalculated every time the metering logic starts.

6.7 Overvoltage Limit Calculated

It reads MPU variables `i_lim`, `i_max` and CE LSBs and sets internal MPU variables that are compared against the CE's voltages.

The internal variables are recalculated every time the metering logic starts.

6.8 Ix_SCALE (LSB translations) Calculated

If a meter has different current sensors, the meter needs to convert the current from the neutral sensor to the current of the main sensor(s).

The CE's `I1_SCALE` is recalculated from `i_maxa` and `i_max_neutral`.

`Ix_SCALE` is a calibration value that can be saved, and can be modified and saved. If there is no saved calibration, this is calculated to make the default. If there is a saved calibration, it is not automatically calculated at start.

If the CE does not perform LSB translation (i.e. it has no `Ix_SCALE` calibration variable), the MPU's internal variables to translate the LSBs are recalculated automatically every time the metering logic starts or by command CLC.

6.9 Changing the pre-amplifier (pre_en) bit

This is bit 8 of the `ADC_VREF_CNTL` (0x4000_D020). The pre-amplifier enable bit. Refer to the Hardware Reference Manual for details. The demo code's default value for this bit is 1. RM command can be used to

set/clear this bit from the SFR. When this bit is set to 0, the `i_max` needs to be recalculated and set by the user. The CLC command will complete the change with the recalculation of other meter configuration such as the `WRATE`. The new variables then will be saved to flash for the next system reset to be in effect.

7 Prototyping LCD operation with the Demo code

1. Most meters need to display a sequence of display items. The Demo code can prototype this with the bit mask `lcd_bit`, a calibration MPU variable that can be found with the CLI help command `"?) "` and set with the CLI command `" ) <number>="`. Setting `lcd_bit` to a nonzero value will select the display items to show, one per bit. E.g. if bit 0 is 1, the M0 display is displayed in the rotation. If the calibration is saved, `lcd_bit` operation becomes normal operation, running after every reset.
2. When a meter displays a sequence of displays on its LCD, a common practice uses the display test to mark the start of the sequence. The Eval code can simulate this by including the display test in the bit mask of `lcd_bit`.
3. The Eval code displays counts of watt-hours, var-hours, etc. This is good for demonstration because they change quickly. But, it is impractical for a meter prototype. To modify the Eval code to display KWh and KVARh, use the command `" ) 1=1 "` and then save the calibration. This selection demonstrates how to do production meter displays.
4. To display megajoules (the customary SI unit for power meters), modify the display code to multiply Wh by 36 and divide by 10000.

8 Building the Demo Code

8.1 For ARM's Keil ARMCC:

The builds are in `\SY7T108\Projects\Demo\ARM_Keil\*.uvproj`

Double click on a .uvproj file; The Keil ARMCC compiler will start, and build the project.

The projects are named: `<part no. prefix>_<meter equation>_<sensor>.uvproj`

e.g. `SY7T108_1p2w_shunt_ct.uvproj`

8.2 For Eclipse/GCC:

TBD

8.3 For Rowley's ARM GCC:

The builds are in `\SY7T108\Projects\Demo\Rowley\*.hzip`

Double click on a .hzip file; The Rowley compiler will start, and build the project.

The projects are named: `SY7T<number>_<meter equation>_<sensor>.hzip`

e.g. `SY7T108_1p2w_shunt_ct.hzip`

9 Demo Application Code

For the overall organization of the source code, the memory map, sample vectors, and global data structures, please see the ARM Meter SDK Users' Guide.

The APIs of the high-level metrology code are in `SY7T108\Projects\Demo\include\*.h`

The high-level metrology code is in `SY7T108\projects\Demo\src\<module>\<module>.c`

9.1 Demo Code Table

Table 25: ARM SDK API files

Driver	Purpose
afe.h	The analog front end code provides most of the calculations describes in the CE reference manual in pretested code. This is a polymorphic interface, so that the code can vary by metering equation.
cli.h	This command line interface is used to demonstrate and calibrate the meters. It is called in only two places in the main code, (<code>cli_init()</code> and <code>cli()</code>) so that it is easy to remove. For quick experiments, a stub is provided. The CLI and help text together use up about 19KiB of flash space.
cal_data.c.h	.h defines the calibration data structure for this code. .c contains the factory default calibration, used if the stored calibration has a bad CRC. These are application code so that unused or unnecessary items can be removed. In the library, <code>cal_eeprom.c</code> and <code>cal_flash.c</code> provide the mechanism to save and restore the calibration.
cli_autoreport.h	This is the API for the autorun code. The code can run on each accumulation interval and report meter data via a serial interface. The output can be captured by a terminal emulator program to form a comma-separated data file that can be imported into a spreadsheet program. The data fields of each record are the accumulation interval index, V A, I A, W A, Wh A, VAR A, VARh A, V B, I B, W A, Wh A, VAR A, VARh A. The API enable and disable routines are called by <code>sercX.c</code> line-editing logic when it receives <code>ctrl_x</code> and <code>ctrl_y</code>. The "run" routine is called by <code>cli_run()</code> when the cli has no output.
cli_display.h	This API is for a simulated front panel that uses the same I/O as the command-line interface. The advantage is that a meter, or meter-like device, does not need to have an LCD. It's also convenient in the lab, or to operate the meter over the Internet.
help.h	Is a list of help strings available in the on-line help text. The CLI and help text together use up about 19KBytes of flash space. The help text can be removed from a build by clearing the 'HELP' symbol in the <code>options.h</code> file.
LCD_BQ14053*.h	This encodes the LCD glass model BQ14053 The .c file encodes the tables. The .h file has the table definitions.

Driver	Purpose
lcd_BQ14053_1_phase.c lcd_strings.h	This is high-level LCD display logic for a 1-phase meter with LCD glass model BQ14053. These call APIs from <code>lcd.h</code> . A display number in the global variable <code>lcd_idx</code> selects which display will show on the LCD. BQ14053 can display 1 8-digit number or text. These display drivers have exactly the same high-level calls for any LCD glass. <code>lcd_strings.h</code> is a list of strings used for the high-level LCD code.
log_events.h	This describes the log events that can be logged in this code. <code>log.c</code> and <code>log.h</code> in the library provide the mechanism to log events. This file defines the log status bits in (4).
main.h	The initialization and main loop.
mapeb1.h	Copy this file, and change it for your meter's PCB. This file describes the items on the printed circuit board: The presence of batteries, usage of pins, and it also includes the correct LCD description.
options.h	This file distributes global symbols to all of the Demo code and drivers. For example, it provides device register names and addresses, PCB descriptions, etc. It also includes include files that are needed in almost every .c file: <code>stdint.h</code> , <code>stdbool.h</code> , etc.
version.h	The Demo code can print versions for its reusable modules. This file gives access to these version strings in <code>version.c</code>

9.2 Application Code Walk-Through

This section should be read with a PC that has the code's source code installed. This section should help users to navigate through the files, to learn by finding and changing the code, not just learn theory.

9.2.1 Main (Demo\Src\main\main.c)

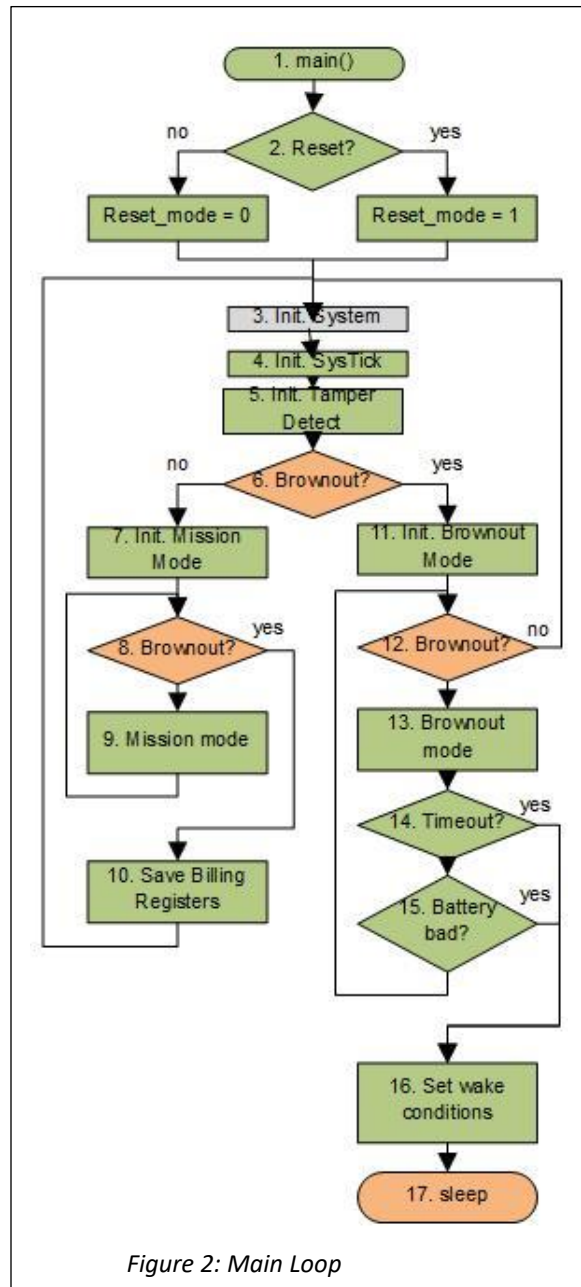


Figure 2: Main Loop

1. `main()` in `Demo\Src\main\main.c` selects and runs a main loop for the current power source. If a CRC of the flash contents is needed, this is the appropriate location.
2. Next, the code tests for reset. This lets the firmware distinguish battery modes from reset. The reset flag is cleared at the end of the operation loops. The Demo code also records the exact values of the `SYS->WAKE_SRC` and `SYS->WAKE` registers so that the command line interface can be used to read these later (this is useful for debugging, experiments and demonstrations; Production meter code need not do this.)
3. The code then initializes the hardware by calling `SystemInit()` in `Sys\system_<SY7_device>.c`. This is a low-level unshared register-level driver. This is not usually necessary, because a reset will clear these registers. The ARMCC startup calls this code earlier. But, in GCC code, an EMI or an accidental start-up, these registers can be in unknown states. Likewise, if waking from a battery mode, registers can have state from the previous operation. This safety code forces the hardware into known states. A crucial part of this is that `SystemInit()` disables all interrupts, sets them to the lowest priority, clears all interrupt flags, then calls an exit-interrupt routine in `startup_<device>.asm` to clear interrupts. This handles a rare situation in which EMI or rarely-executed code causes the reset vector to be executed when there is an outstanding high-priority interrupt. If not handled, this case will hang lower-priority interrupts. It does enough return-from-interrupts instructions to clear the interrupt controller. `SystemInit()` also enables the fuse fail interrupt. The fuse-fail interrupt runs if the factory-calibrated values are misread by internal hardware or become corrupt. In this case the firmware must arrange to reread these values, either by initiating a fuse-reread using the flash hardware (as done in the Demo code's fuse interrupt) or code may perform a reset (e.g. with the code `SYS->MOD_CNTL_b.reset = 1`). One reason the fuse registers can become corrupt is by EMI.
4. It sets up the SysTick interrupt to 5Hz. This wakes up the main and brownout loop several times per second so that they update the software timers.

5. It then sets-up the tamper system, and/or it polls the tamper GPIOs. In the Demo PCB, the case-open and terminal-block-open tampers are attached to external interrupts. The tamper hardware is initialized after system hardware is in a known state.
6. It calls `pwrmode_is_brownout()` a shared driver routine to test whether Vcc is available and the system can run normally, or the whether the system should switch to a battery mode.

7. If there is Vcc, it initializes mission mode. This sets up the analog front end and meter output I/O.
8. It tests whether it should remain in mission mode.
9. `main_mission_mode()` is the conventional meter main loop operating in the mission mode, powered from Vcc (i.e. powered from AC mains, not powered from a battery.)
10. After it leaves mission mode, it saves the billing registers with a call to `meter_save()`.
11. If the mains power fails, i.e. it is in brownout mode, running from battery, it enters a loop to handle the mode in which the meter runs from the battery.
12. The brownout mode loop tests whether it should remain in brownout mode. If mains power has returned, it returns to the mission mode loop (5, then 6) the mode for normal power from AC mains.
13. `main_brownout_mode()`, is an alternative main loop operating in the battery-powered brownout mode. In this mode, the meter is powered from the battery (Vbat), and should conserve energy. The analog front end is shut down because it is powered from V3P3SYS (i.e. Vcc), and therefore inaccurate. In the Demo code, this mode permits the command line interface to run, which can be used by a customer engineer to explore the behavior of the SoC when powered from a battery. Many production meters would simply enter sleep mode at this point.
14. It detects a customer engineer by serial I/O and a timer. If a command line is entered, the timer is pushed off for a time.
15. If the battery power fails, the system will go to sleep in any case.
16. It sets the register `SYS->WAKE` to set the conditions that will cause it to wake form sleep. Of course, it wakes if Vcc returns. It also wakes if a tamper is detected.
17. It sleeps to reduce battery power.

9.2.2 main_mission_mode_init() (Demo\Src\main\main.c)

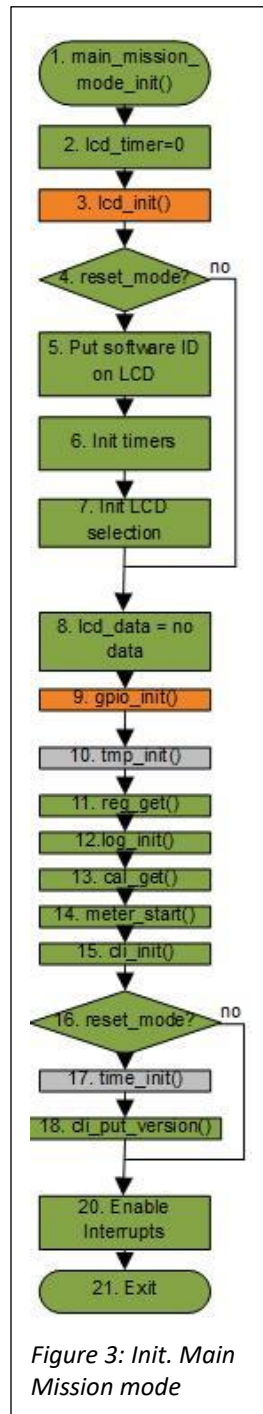


Figure 3: Init. Main Mission mode

1. This routine initializes the system for full power “mission mode” normal operation. The order of initialization is important because the later initialization operations use information from the earlier operations.

2. The LCD timer variable is cleared. The LCD timer lets the high-level LCD code, `lcd_update()`, in `projects \demo \src \LCD \lcd <glass_name>_lphase_iso.c` present a scrolling LCD display for a 1-phase meter using ISO terms. (`global.cal.lcd_bit` is a bit-mask with a bit that can be set for each desired numbered display. If it is zero, the display does not scroll. It is a cli variable, (see the MPU Parameters document) and can be set with the cli and saved as part of the configuration, to produce a prototype system.)

3. The call to `lcd_init()`, initializes the LCD. It sets all the bits in `LCD->MAPH` and `LCD->MAPL`. Later, any call to gpio routines (see `drivers\include\gpio.h`) for these pins will abort if these bits are set.

4. If the code has just reset...

5. It displays the software’s version number on the LCD by calling `lcd_put_sw_id()`.

6. It also arranges to start the timing logic by initializing the comparison variables `global.misc.last_sec`, etc.

7. The LCD display is written when the selected display changes (as well as when there is new data,) so at start-up to make the display appear, the “previous display selection index”, `lcd_idx_old`, has to be set to a value (0xff) that will be different from any possible value of `lcd_idx`.

8. `lcd_data` is marked false, so that the software version ID on the LCD will remain visible until the first valid meter data arrives.

9. `gpio_init()` checks the `LCD->MAP` registers, and writes to the `DIO[]` registers to force non-LCD pins to the bit pattern of “alternate”, i.e. not being used by the LCD system in any way. This enables the ARM Cortex M0 I/O system for all the pins than are not in use for the LCD. This is safety code, used to set-up bits that usually enter the correct state from reset. So, why is this a good idea? Imagine that EMI corrupts the LCD system. Besides LCD, the LCD system also provides an entire alternative way to program pins as GPIOs. Since the LCD system is a layer wrapped around the outside of the ARM Cortex M0 I/O pins, the LCD system has precedence over the other I/O. `gpio_init()` is cheap insurance that puts the DIO in the expected state, permitting ARM Cortex M0 I/O to use the pins as DIOs.

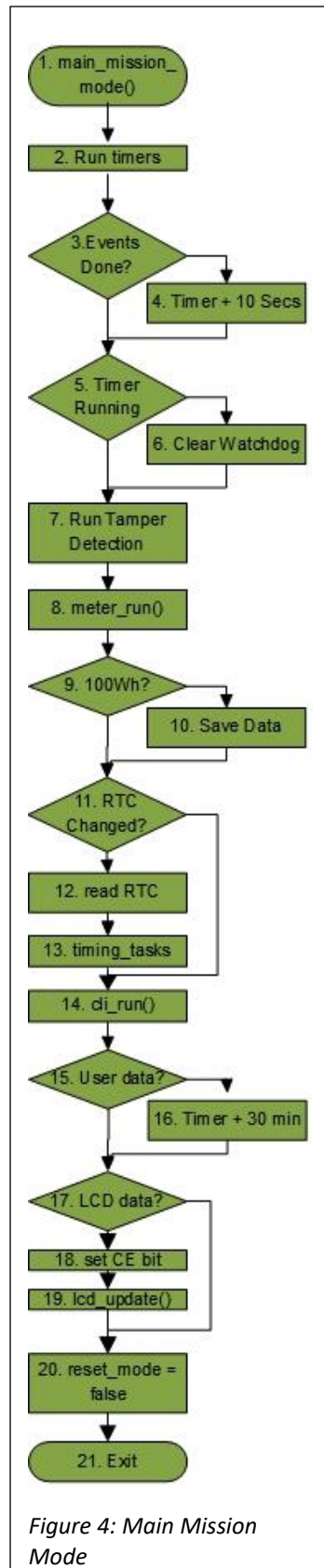
10. The code then sets-up and makes the first temperature reading by calling `tmp_init()`. This is a register-level driver in `drivers\include\tmp.h`

11. `reg_get()` provides a verified valid copy of most of the dynamic, nonvolatile data in the meter. It has to be called before the interrupts are enabled, or else the sag interrupt could save an invalid copy of the billing registers. There is an EEPROM version, also a version that stores the data in flash with wear-levelling. It stores money, i.e. the billing registers, so it has to be very reliable. It’s also called in battery modes, so it tries to avoid power-hungry operations, especially unnecessary reads of the EEPROM. It uses a CRC to check several sets of the billing registers, starting with one in battery-backed RAM. (The RAM copy is updated once per second. Also, accessing the RAM copy uses the least energy.) It takes the newest

copy that has a valid CRC, a matching software version hash and the most recent save count. The software version hash assures that when new software is loaded, the old billing registers are not used (the C structure might be different). If no copy is valid, it creates a default value, and logs an error. (The code keeps running.) The data includes the billing registers, but it also has log data, a recent copy of the real time from the clock and status telling the daylight savings time state and whether the real-time clock has been set. `reg_get()` also overwrites the pulse counters in the CE, setting them to the last-saved valid values. At this point, however, the CE is not yet initialized, and the initialization will later overwrite these.

Therefore, `meter_start()` later copies the pulse counters again, after the CE is initialized, but before it is started.

12. `log_init()` initializes the error system. It adds any events from the start-up to the long-term errors stored in the nonvolatile meter data from `reg_get()`. It must be called after `reg_get()`.
13. `cal_get()` reads the last saved calibration, or if that has a bad CRC, the factory default. There is a tested version that stores the calibration in flash.
14. The call to `meter_start()` sets up the software to use the AFE (analog front end) data, then calls the CE's init to initialize the AFE hardware. It must be called after `reg_get()` and `cal_get()` because it needs the current billing data and the calibration in order to start metering. The AFE must be initialized before the version is displayed, because the version logic shows the CE code string from the CE. This also starts the high-level demand-metering logic. The demand metering logic runs in the `timing_tasks()`, each time a demand interval starts.
15. At `cli_init()`, the code starts the serial port(s). The CLI uses about 19K, and a serial or slave-SPI port. Many customers don't need or want the CLI. Feel free to remove every call that begins "`cli_`" and then remove the CLI files from the build.
16. If there was a reset...
17. `time_init()` tests the RTC. If the value in the RTC is invalid, or there was a battery failure, the RTC is marked "unset." If no valid copy is available (e.g. from the call to `reg_get()`), the RTC is set to a bad value as well.
18. `cli_put_version()` starts the serial start-up message. Nothing is actually sent until the interrupts are enabled. A bit later it sets `lcd_data` to false so that the version message stays on LCD until metering data replaces it.
19. The last step is to enable interrupts. At this point, everything with an interrupt has to be ready: Billing register data, in case there is a sag interrupt (the CE pulses YPULSE, and the MPU sets up the YPULSE interrupt to detect AC mains failure.), the analog front end, the serial ports, e.g. the CLI, and any other interrupts.



9.2.3 main_mission_mode() (Demo\Src\main\main.c)

1. This routine is called by the main loop. It runs the meter as long as the meter has power from AC mains.
2. It runs the software timer's main logic, `stm_run()`, from `library\src\stm\stm.c`
3. It checks to see if needed periodic events are done. The needed events are flagged as bits set in the bit-mask `global.misc.wd_want`, the events detected are bits set in `global.misc.wd_set`. The firmware checks several periodic software events.
4. If the needed events actually occurred, a software timer is reset to ten seconds. Why use a timer? The clear occurs at a time that is not synchronized with any of the events. So, it can take several seconds for all the events to occur.
5. If the software timer is running...
6. The hardware's software watchdog is reset.
7. Tamper detection resets the interrupts or polls the GPIOs if the tamper detection doesn't use interrupts. The pins are defined in the PCB feature-map file, e.g. `mapeb11_1.h`, included by `options.h`
8. `meter_run()` actually performs the meter operation. It calls routines from `afe.c` to read hardware metering data, perform creep, sum energy, etc. It returns a Boolean that is true if there is new data for the LCD to display.
9. If 100 Wh have accumulated, then...
10. Save the billing data. It also saves the billing data on power failure, and once per hour.
11. "`if (RTC->TIME != last_sec)`" code is true if a second has passed. `RTC->TIME` is an SFR that changes once per second. The ARM Cortex M0 reads it with a single instruction, so it does not turn over.
12. If a second has passed, it reads the RTC, and...
13. Runs timed tasks. This handles tasks that must occur once per second, once per minute, once per hour, etc. It is shared with the brownout mode. (See below).
14. `cli_run()` runs the command line interface. It returns a nonzero result if any commands were typed.
15. If commands were typed...
16. Extend the user software timer by ½ hour. As long as the user timer is nonzero, the firmware expects a user to be present.
17. If there is LCD data, either from `meter_run()` or if the user sent CLI commands to see different LCD displays... then
18. Indicate that the CE completed a cycle, so that the watchdog timer can be reset. (The firmware checks several periodic software events.)
19. Update the LCD. From `\Projects\Demo\Src\LCD\lcd_<glass name>_1phase_iso.c`
20. It clears the reset mode flag, so that after this, the meter is no longer initializing.
21. It returns to the main loop in `main()`.

9.2.4 main_timing_tasks() (Demo\Src\main\main.c)

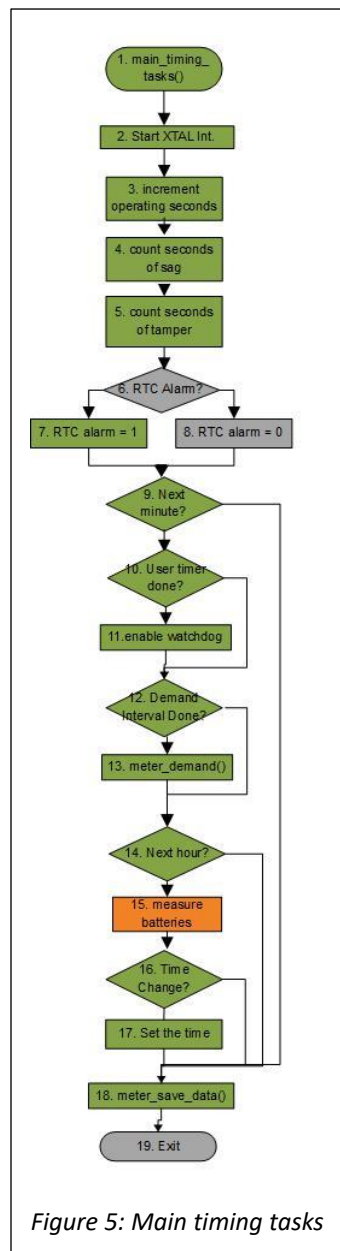


Figure 5: Main timing tasks

1. "main_timing_tasks()" checks for once-per-minute, once-per-hour and once-per day events. Note that the ARM Cortex M0 code does not interrupt at these times. The timings for most system events are from the `RTC->TIME` and `RTC->SUB` registers of the RTC. These registers run at the same speed no matter what speed the Cortex runs.
2. The crystal interrupt is restarted if the crystal is OK.
3. Count operating seconds. (Divide to get hours, tenths of hours, etc.) Operating seconds counts how long the meter has been on, since power-up. It's nonvolatile, saved and restored with the billing registers.
4. Then, it counts seconds of sag and...
5. Counts seconds of tampering.
6. It checks, the alarm clock and...
7. sets it
8. or resets it.
9. If this second starts the next minute...
10. If the user timer expired, i.e. the user is not present...
11. Then enable the watchdog on the CE.
12. If the minute of the hour falls on the start of a demand interval (i.e. modulo of the demand interval is zero.)... (Since this is Demo code, the default demand interval (in the calibration structure--it can be changed) is only two minutes.)
13. Call `meter_demand()` in `SY7T108\ Projects\ Demo\ Src\ meter\ meter.c` to perform high level meter demand processing. The demand power data is summed to a power register once per second in `meter_run()`. The power register cannot overflow or underflow like a floating-point number. At the start of the next demand interval, the demand power register is converted to a floating-point count of CE energy units, and divided by the number of accumulation intervals. This gives a number scaled the same as one accumulation interval, so the standard watt-display calculation can use it. After getting the demand, it compares it to the biggest demand so far. If the new demand is bigger, the code saves the new demand, and the time that the interval started. This demand logic copes well. For example, the meter normally doesn't start exactly on a demand interval. This logic resynchronizes because it uses the minutes past the hour and runs when the remainder is zero. It also calculates the demand accurately in a short starting-interval, and records the actual time that a short interval started.
14. If it's the next hour...
15. Measure the batteries. This is infrequent enough that a battery failure is noticed in a usefully short interval. The duty cycle is low, so the batteries can be measured with a 100uA current. This is a realistic overload, and also "burns off" any oxide. Yet, it's infrequent enough not to affect battery life measurably. The
16. If there is a change to or from daylight savings time...
17. Set the RTC to the changed time. The local time is stored in the RTC hardware. This saves power, CPU time and software complexity. (It is not like a POSIX-compliant system.)
18. Save the billing registers, in case of a failure of the register save when the power fails. The register save logic is designed to provide at least a million writes in a typical EEPROM, so it will last at least 114 years. After a register save, the save is prevented until there is new data.
19. This exits back to the main loop.

PCB_HAS_VBAT and PCB_HAS_VBAT_RTC build switches are defined in the `mapeb11_1.h` file to match the Demo PCB.

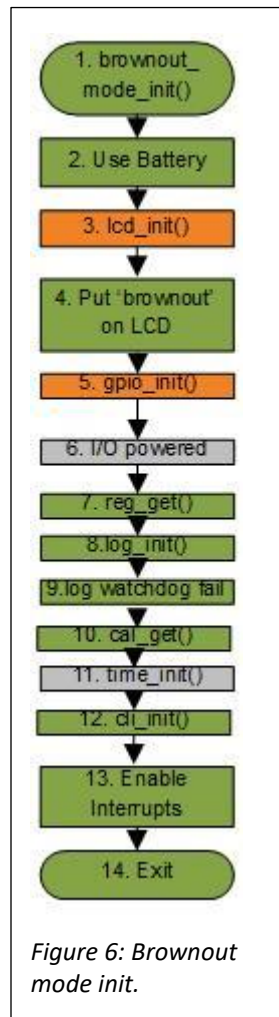
9.2.5 main_brownout_mode_init() (Project\Demo\Src\main\main.c)

Back in `main()`, in `main.c`, the battery modes start when the main 3.3V power falls below 3.0V. At this voltage, the meter's analog front end becomes inaccurate. This indicates that the mains voltage is sagging, and the meter will soon need to run from battery power.

At this point, `pwrmode_is_brownout()` returns true, and the mission mode loop exits.

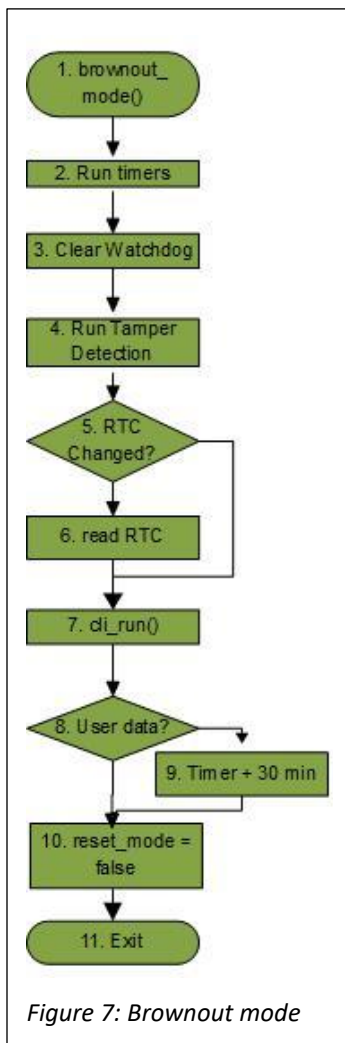
After `main_mission_mode()` exits, the main loop saves the billing data.

The main loop restarts, and reinitializes the IC.



1. Then it calls `main_brownout_mode_init()`.
2. It adapts the PCB to battery operation; This varies by the application, but the demo is typical. For the Demo code:
 - a. It stops the analog front end. This saves power. Also, in brownout mode, since most of the analog items are powered from V3P3A, they are not accurate.
 - b. The RTC's temperature measurement is changed to run from the battery, because the RTC and its temperature measurement run continuously. Also,
 - c. It sets the MPU's clock speed to the recommended speed for the PCB's battery operation. Usually this is a much lower speed. In some PCB designs it might be faster, to minimize leakage by finishing the transition to sleep faster.
3. The LCD is initialized. The brownout mode has to be designed to run from reset, as well as during a transition from mains power.
4. It displays "brownout" on the LCD, so the Demo operation is clear.
5. `gpio_init()` initializes the `DIO[]` array. This is done after the LCD is initialized, so that only the `DIO[]` pins unused by the LCD are initialized.
6. The I/O power (eeprom and pulse on the M11's Demo PCB) is enabled. This is so that the EEPROM works, so the billing data and calibration can be read. These are read so that the command-line interface can be fully functional in the brownout mode. If the brownout mode had no CLI, there would be no need for this.
7. `reg_get()` reads the billing registers from the nonvolatile storage, nonvolatile RAM, EEPROM or flash. To save energy, if the copy in the nonvolatile RAM has a good CRC, the EEPROM or flash is not accessed.
8. `log_init()` initializes the log, merging the start-up log data with the nonvolatile log data from the billing data.
9. If there was a watchdog reset, that is logged.
10. `cal_get()` reads the calibration data from nonvolatile storage. The nonvolatile RAM does not have enough space to store the calibration data, so this is always from EEPROM or flash.
11. `time_init()` starts the RTC reliably, even if there was EMI corruption, or dropouts of power or clock. If this takes too much time or power, the result of `rtc_init()` can be logged to detect an unset clock.
12. `cli_init()` starts the command line interface. At this point, all the data and items needed by the CLI are present.
13. Interrupts are enabled, enabling the I/O for the command line interface.
14. It returns to the brownout mode section of the main loop.

9.2.6 main_brownout_mode() (Demo\Src\main\main.c)



1. In the main loop, to do the brownout power mode, run `main_brownout_mode()`
2. Run the software timers.
3. Clear (or reset or feed) the hardware watchdog.
4. Detect tamperers.
5. If the RTC has changed...
6. Read the time from the RTC.
7. Run the command line interface (CLI)
8. If something was typed several times in a few seconds (e.g. first 3, then 15 seconds)...
9. Then stay awake (i.e. not in sleep mode) for 30 minutes so that the user can experiment with the battery modes. The unit wakes when the battery is inserted, so if there is no user present, it should sleep.
10. Mark that reset operations are done.
11. Exit

The main loop checks to see if the user timer has expired. If it has, then it sleeps to save the battery.

Logic to sleep is required to prevent the battery from depletion.

9.2.7 DSP transfer done (XFER_I_N) Interrupt (\drivers\src\ce\ce.c)

The CE interrupt vector (there is only one) is found in SY7T108 \Drivers \Src \sys \startup_SY7T108.asm.

The XFER_I_N interrupt is requested by the CE at the conclusion of every accumulation interval. This enables the XFER busy interrupt in CE_Handler() in ce\ce.c. This clears a flag. Later, the main loop wakes from idle, and calls meter_run() in meter\meter.c meter_run() calls afe_status() in afe\afe_0_1.c to check the flag. Then meter_run() will process the new data from the CE and analog front end (AFE). For other displays of meter data, (e.g. for the LCD), meter_run() returns true, indicating that there is new data. This causes the LCD to update. The LCD routines fetch appropriate data, directly from the AFE if needed.

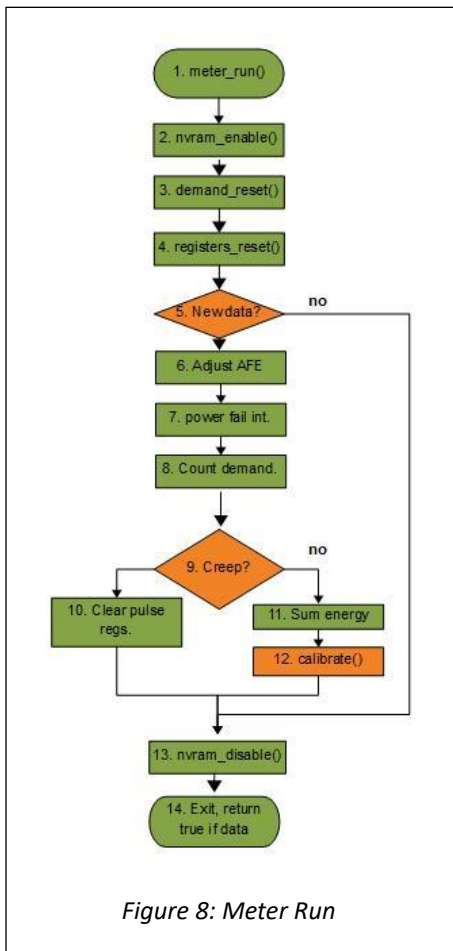


Figure 8: Meter Run

9.2.8 meter_run() (Demo\Src\meter\meter.c)

1. `meter_run()` in `library\src\meter\meter_1p2w.c` actually does the high-level meter logic.
2. `nvram_enable()` enables the nonvolatile RAM code. It's normally disabled to protect the billing data from EMI.
3. `meter_demand_reset()` checks if the demand should be cleared. It has to synchronize with the main meter operations.
4. `meter_registers_reset()` checks to see if a user asked for the power registers to be cleared. It's run here because it has to be synchronized with the main meter operations.

New meter data happens about once per second

5. To find if there is new data, it calls `afe_status()` (in `Library\afe\afe*.c`). In `afe*.c` flag "`afe_busy`" is cleared by a CE interrupt about once per second, when the CE has more DSP data to report.
6. If there is new data... it adjusts the analog front end for temperature.
7. it enables the AC mains power failure detection interrupt from the CE: `ce_int_enable( YPLS_I, meter_save_data );`
8. It counts the accumulation intervals for the demand calculations.
9. If either the voltage and current are smaller than the creep values (global variables from `afe.c`), then the meter is in its noise floor, and the DSP data is invalid, and should not be added to billing registers. Checking both voltage and current is the most accurate, most stable method of managing creep in EMI. The creep test also checks for other status, and logs it.
10. In creep... It clears the pulse inputs to the CE so that they don't flash.
11. Else, if the data is valid, it adds up the billing registers by calling `meter_sum_data()`. In this routine, the billing registers are summed using `add()`, a special arithmetic routine that adds CE data to a 32-bit power register without overflow, underflow or other loss of precision. It is in `Meter\mmath.c`. It also saves the billing data: `reg_crc()` is in `library\src\reg\reg.c`. It updates the double-buffered billing registers. The billing registers are double-buffered, so if there is a power failure, one billing register structure always has correct data and a valid CRC. This code also sets a flag `new_data` so that `meter_save()` will save data.
12. It runs the autocalibration's data collection routine `calibrate()`, the autocalibration system (in `SY7T108\drivers\ce<name>\autocalce<name>.c`). If autocalibration is in progress, it steps through a state machine on each accumulation interval. It waits for the system to stabilize, collects data, then adjusts the gains and phases. If calibration is disabled, it does nothing.
13. NVRAM is disabled so that the billing data in NVRAM can't be corrupted by EMI.
14. It returns a flag that is true if there was new DSP data. This tells the high-level code in the main loop to update displays.

9.2.9 Meter (...\\library\\src\\meter\\meter.c)

This folder has a simplified piece of meter code, showing basic logic for calling the AFE to do utility metering.

Input

It gets data from the analog front end (AFE), calling code from `afe.c`

Process

After reset, `meter_start()` should be called.

After that `meter_run()` can be called continually from the main loop. It detects new data (search for `new_meter_data`), then it applies creep detection and adds the new data to the billing registers using the routines from `mmath.h`. After the summation, it makes a secure, CRCed copy of the billing registers (see `reg_crc()`) then saves a copy to nonvolatile RAM.

When the main loop or the sag interrupt detects a power failure, it calls `meter_save()` to save a copy of the billing registers. The main loop of the Demo code also saves the billing registers once per hour, and every 100 Watt-Hours.

Output

`Meter_run()` returns a nonzero value to indicate that the meter has new data. The main loop uses that to display the changes on the LCD.

When the main loop or the sag interrupt detects a power failure, it calls `meter_save()` to save a copy of the billing registers. The main loop of the Demo code also saves the billing registers once per hour, and every 100 Watt-Hours.

The `reg.c` logic saves multiple copies of the billing registers, and the Demo calls the save code from a number of events. Since there are two copies of billing registers in RAM, one of them is always saveable. This is essential, because a power-failure cannot wait.

Another particular issue is that the billing register data has to be intact and in RAM before interrupts are enabled. Otherwise a power failure can save bad data.

Datastructure

The 32-bit major counters of the billing registers (Wh, VARh, etc.) are in `global.reg`, defined by `reg.h` and `global.h`. These are preserved in a power failure.

The minor counters of each billing register, roughly equivalent to the position of a disk in a mechanical meter, use CE units, identical to the units of the CE register `W0SUM_X` (See the CE reference manual.) The minor counters are in `global.misc`, defined by `global.h`. They are not preserved if there is a power failure.

9.2.10 lcd_update() (Demo\\Src\\LCD\\lcd_1phase.c)

Back in the main loop, the code “`if (lcd_data)`” checks if there is new data for the LCD.

In the main loop, the routine `lcd_update()` is from `<device name>\\ projects\\ Demo\\ src\\ LCD\\ lcd_<display name>_1phase.c`. This routine displays meter data that it gets from `afe\\afe.c`. It also sets annunciator segments that describe the meter’s status.

For example, it may display a battery failure segment or a “lid open” warning. The annunciators differ with the type of LCD-glass, so this code is in the LCD folder, near the tables that define the LCD glass. Also, ISO and ANSI meters use different terms on the front panel, and different metering equations have different displays. `lcd_update()` varies to cope with all of those high-level application particulars.

If the display changes, the code clears the LCD completely, just in case any segments have been accidentally turned on.

Then, the changed data is displayed on both the serial line, and the LCD screen.

If the button is held down, the next display appears. After the last display, it wraps to the first. The displays are in order according to the “M” commands (see the M command table in the MPU parameters document), the imported watt-hours, is the default, followed by M4, exported watt-hours, M5 imported var-hours, etc. up to M22, the LCD test, followed by M0.

The display is updated once per accumulation interval. `global.cal.lcd_bit` is a bit mask telling which displays should display in an automatic rotation. For example, 0x0009 will display M0, the meter ID (because bit 0 is set), and ARM CORTEX M0, the imported watt-hours, (because bit 3 is set). If `lcd_bit` is zero, the meter display does not rotate.

10 Appendices

10.1A.1 Acronyms

AC	Alternating Current – current with changing polarity
ADC	Analog to digital converter. This is the basic measurement device of an SoC.
AFE	Analog Front End, the combination of analog electronics, ADCs, CE and ARM CORTEX M0.
AMI	Automated metering infrastructure. Usually automated meter reading with remote disconnect.
AMR	Automated Meter Reading, usually performed via an optical port or modem
ANSI	American National Standardization Institution, part of ISO
ANSI C	C Programming Language, standardized by ANSI in 1983.
API	Application Programming Interface
ARM	ARM Limited, a company selling CPU designs to semiconductor companies.
ARM Cortex M0	A 32-bit pseudo-Von-Neumann architecture CPU designed by ARM Limited.
C	The C Programming Language, as defined by ANSI C99
CE	Computation Engine, the low-level DSP in the SoC.
<CR>	Carriage Return or Enter Key on PC Keyboard
CMSIS	The Cortex Microcontroller Software Interface Standard, a vendor-independent abstraction layer.
COM	Communication Port, usually in a PC.
Cortex	See ARM Cortex M0.
CPU	Central Processor Unit
DC	Direct Current
DSP	Digital signal processor, or processing.
EMI	Electromagnetic interference, e.g. a nearby lightning strike or arc-welder.
EEPROM	Electrically Erasable PROM
EV Kit	An evaluation kit. An evaluation Board in a box with other items.
FLAG	An international protocol for reading of meters using an optical port, initially developed by Ferranti and Landis & Gyr
Flash	Rewriteable code memory. It is erased in “pages,” normally 512 bytes long. It only permits a limited number of writes, usually 100,000. Like all flash memory, when it fails from too many writes, the entire flash memory is shorted out and in this case, the entire SoC must be replaced.
GB	Gigabyte(s)
Handler	An interrupt service routine.
ICE	In-Circuit Emulator
IDE	Integrated Development Environment – usually a combination of editor, compiler, assembler, linker, debugger, ICE
IEC	International Electrotechnical Commission (Geneva, Switzerland)

INT	Interrupt
ISO	International Standards Organization
ISR	Interrupt Service Routine
KB	Kilobyte(s) – 1,024 bytes
LPM	Low Power Metering, a mode in the SoC used to operate with very little power. See the discussion in the batmode module for details.
LCD	Liquid Crystal Display
<LF>	Line-feed character
LSB	Least Significant Bit
VLCT	Voltage Line (is) Cut Tamper.
MB	Megabyte(s) – 1,024 kilobytes
MPU	Microprocessor or microcontroller Unit- the ARM Cortex M0 in the IC.
MSB	Most Significant Bit
NV	Non-Volatile
NVM	Nonvolatile memory
NVRAM	Nonvolatile memory
PC	Personal Computer
PROM	Programmable read only memory. See “flash.”
PSU	Power Supply Unit
PSW	Program Status Word
RAM	Random Access Memory
ROM	Read Only Memory
RTD	Resistance Thermal Detector; a resistor designed to sense temperature.
SDK	Software Development Kit
SFR	Special Function Register
TOU	Time-of-Use (variable metering tariffs, usually based on time of day)
USB	Universal Serial Bus
VA	Volt-Amperes (apparent power unit)
Vah	Volt-Ampere-Hour (apparent energy unit)
VAR	Volt-Amperes, Reactive (reactive power unit)
VARh	Volt-Amperes-Reactive-Hour (reactive energy unit)
W	Watt (power unit)
WD	Watchdog
WDT	Watchdog timer
Wh	Watt-Hour (energy unit)

11 Revision History

Note: Revision of this document follows the revision of the firmware, so an update that duplicates the revision is correct.

Table 26: Revision History

Revision	Date	Description
3.11	January 18, 2021	Removed trim write and refresh commands. Documented cal_data.c, .h and log_events.h, which adapt library routines to a particular project. Changed names to SY7T108.
3.1	August 25, 2020	Removed WFI implementation from the code and from this document. Removed pre_en MPU parameter, updated descriptions ver_hash, reg.ver_hash, last_wake_src, and last_wake MPU parameters. Added note to MPU parameter address table, that certain parameters do not take effect until command CLC runs, or a calibration is saved and restored Added section “Changing the pre-amplifier (pre_en) bit” to describe how to change the pre_en bit.
3.1	August 14, 2020	Added explanatory text to battery mode commands: BRn, BL, BS, BT. Added notes to “)” and “CLC” referring to use of CLC to adjust MPU parameters.
3.1	May 21, 2020	Described the information fields of the I command.
3.0	April 1, 2020	Reworked headers, table of contents. Rolled revision to match firmware. No technical changes.
2.1	January 7, 2020	Updated commands: BT, DA (removed), P, RA, RM, RTU, RTM, TW, TFR, Z4 Changed all ‘Eval’ references to ‘Demo’. Fixed a few small issues with incorrect address references. Moved the RM table to be in between RA and RTC
1.1	September 26, 2018	Added use of WFI to CLI commands (meaning of “W” command changed.) and to documentation of the main loop, mission mode and brownout mode.

Revision	Date	Description
		Documented the current order of the errors in log.h Removed CRCs from MPU parameter table. CRCs changed to a byte array, and displaying them became inconvenient.
1.0	June 5, 2018	Changed “demo” code to “eval” code. Rolled revision to 1.0 for packaging.
0.5	May 24, 2018	Added table describing the “D” command.
0.4	May 18, 2018	Removed brief discussion of SDK and data structures, by reviewer request. Updated the log table to the latest as-built.
0.3	May 7, 2018	Reviewed, name changes to devices, split into application and SDK.
0.2	January 24, 2018	Modified to as-built.
0.1	November 30, 2017	Initial Release.